

C PROGRAMMING

*Objectif de ce document ; guide de survie en C pour un débutant qui n'a jamais trop touché.
Idéalement quelques idées de programmation.*

Objectif de l'auteur du doc (Benjamin Suger) : expliqué à ma manière les concepts de base pour mieux les maîtriser, me pousser à faire des recherches plus poussées sur certains sujets, etc....

*Source ; Openclassroom, Cours du Dr Chuck, HackerNews, Google, zestedesavoir,
<https://www.cc4e.com/index.php>, livre C programming, youtube....*

*V1 => V2 ; changement du plan (quitte la structure d'openclassroom), ajout de précision sur
pointers, array, structure, GDB, enum, union, préprocesseurs, ajout d'exemple, mes vidéos
youtube (d'autres en janvier), décision de ne pas ajouter plus de précision sur makefile (video
incoming + lien vers une ressource), .o object files expliqué dans vidéo sur makefile*

*Version 2 du document. Potentiellement des mises à jours un jour si j'ai le temps d'ajouter du
contenu. Ajout à faire ;*

- plus d'exemple sur les variables constantes => utilisation pratique>*
- exemple de #define*
- ternaire explication + exemples*
- Linked list*

C est un langage de bas niveau. C'est-à-dire qu'il s'éloigne de notre langage qu'on utilise pour communiquer mais il est plus proche de celui de la machine.

D'autres langages de programmation se base sur C. Par exemple Python, qui est un langage de programmation plus haut niveau est lui-même « coder » en C.

Début d'un programme

Dans les lignes du début, il y a `#include <bibliothèque>` c'est pour inclure une librairie/bibliothèque. De base on parle de bibliothèque mais un anglicisme fait que parfois on parle de librairie.

Les bibliothèques contiennent des fonctions qu'on peut utiliser.

On peut en avoir plusieurs d'affilés dans un même programme;

```
#include <stdio.h>
#include <stdlib.h>
```

Si on découpe `stdio.h` ;

- `-std` pour standard
- `-io` pour input output
- `-.h` pour header

C'est donc la bibliothèque qui gère les input et output. On peut voir toutes les fonctions utilisables ici ; <https://www.ibm.com/docs/fr/i/7.5?topic=files-stdioh>

stdlib.h c'est pour « *standard library* »

Fonction main()

Un programme commence « toujours » par la fonction *main*. Par là je veux dire qu'il va exécuter ce qu'il trouve dans main, mais il peut y avoir du code avant et après.

Main qui veut dire que c'est la fonction principale.

(le toujours est entre guillemet, voir plus tard plus de détails sur les fonctions).

On la déclare ainsi ;

```
int main() {  
  
<instructions> ;  
  
return 0 ;  
  
}
```

main() est notre fonction principale. Entre parenthèse il n'y rien dans cette exemple. Ici c'est vide car on a pas de paramètre dans notre fonction, cela peut arriver (voir plus tard).

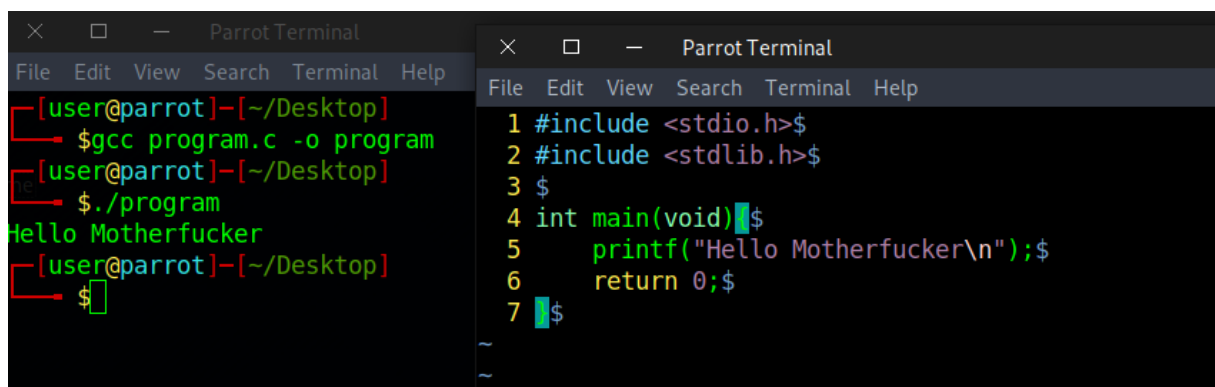
Entre les {} on va mettre les instructions. C'est-à-dire ce que doit faire le programme, chaque instruction se termine par un point-virgule ;

Ici à la fin on fait **return 0 ;** car notre fonction est un **int**. C'est-à-dire que la fonction attend à la fin un *integer* (=nombre entier negatif ou positif +0).

On ne commence pas forcément par **int**, il y a différent types possible.

Exemple d'une fonction de base (décrite) ;

Ici on est sur une VM (virtual machine) avec Parrot OS. J'utilise VIM pour taper mon code.



```
Parrot Terminal
File Edit View Search Terminal Help
[user@parrot]--[~/Desktop]
$ gcc program.c -o program
[user@parrot]--[~/Desktop]
$ ./program
Hello Motherfucker
[user@parrot]--[~/Desktop]
$

Parrot Terminal
File Edit View Search Terminal Help
1 #include <stdio.h>$
2 #include <stdlib.h>$
3 $
4 int main(void)$
5     printf("Hello Motherfucker\n");$
6     return 0;$
7 $
```

A droite ; le programme pour faire écrire « Hello Motherfucker » sur l'écran.

A gauche ; compilation + exécution du programme.

Description sur la gauche ;

Une fois que l'on a écrit notre code, dans program.c, il faut le compiler. *Qu'est-ce que compiler ?*

C'est le moment où on traduit notre code en langage machine. L'ordinateur ne comprenant pas le code C qu'on a fait. L'ordinateur ne comprends que le binaire in fine.

Ici je compile avec gcc, comment marche la commande ?

`gcc <votre_code_en_point_c> -o <le nom que vous souhaitez mettre en place>`

l'option -o n'est pas obligatoire mais ça vous permet de choisir le nom. On peut ajouter plusieurs options -Wall -Werrors etc... Elles ne sont pas obligatoire mais peuvent être requise selon les exercices ou contexte dans lequel vous coder.

Pour lancer le programme on écrit `./<nom_du_programme>` car je suis déjà là où il faut, c'est-à-dire que je suis dans le directory où tout est présent.

```
└─ $ls
program program.c README.license
└─ [user@parrot]~[~/Desktop]
└─ $
```

`ls` dans le shell veut dire *listing*, on voit que j'ai mon `program.c` puis `program` que j'ai compiler.

Lancement depuis un autre directory (à comprendre que c'est l'endroit dans lequel je me trouve avec mon terminal) ;

```
└─ $pwd
/home/user/Documents
└─ [user@parrot]~[~/Documents]
└─ $./../Desktop/program
Hello Motherfucken
└─ [user@parrot]~[~/Documents]
└─ $
```

`Pwd` vous donne votre chemin d'accès, `pwd` = print working directory.

La ligne `./../Desktop/program` me permet de lancer le programme qui est dans `/Desktop/`

Attention potentiellement sur linux => il est possible que vous ne pouvez pas utiliser votre programme avec `./<nom_de_votre_programme>` car vous n'avez pas les permissions.

Vous devez taper `chmod +x <nom_de_votre_programme>`.

A droite ; on a notre code.

Les `#include` nous permet de faire appel aux librairies. Ici particulièrement `#include <stdio.h>` va nous permettre d'utiliser `printf()` qui vient de cette librairie.

`Printf()` est donc une instruction qu'on termine bien par un point-virgule ;

Sur l'utilisation de `printf()`, le string (= chaîne de caractère) doit être entre guillemet. On utilise `\n` pour le saut de ligne pour éviter quelque chose de moche comme ça ;

```
└─ $ ./program
Hello Motherfucker
└─ [user@parrot] - [~/Desktop]
└─ $ ./program
Hello Motherfucker └─ [user@parrot] - [~/Desktop] SANS LE \n
└─ $
```

On finit avec `return 0` ; qui n'est pas obligatoire pour que cela marche, cependant d'après différentes sources cela est recommandé en bonne pratique.

Paramètre du programme

Dans notre exemple, on vient juste de lancer le programme sans paramètre en plus.

Dans l'explication que j'ai pu donner sur `main()` du programme, j'ai précisé que nous ne mettions rien car pas d'argument. Mais qu'est-ce que c'est un argument avec un programme ?

Un argument est un élément qu'on doit préciser en plus pour utiliser notre programme. Prenons l'exemple d'un outil dans linux `cat`.

```
CAT(1)
NAME
  cat - concatenate files and print on the standard output
```

`Cat` permet de lire un fichier et de sortir son contenu dans notre terminal.

Mais pour utiliser `cat` il faut lui préciser un fichier sinon il ne va rien lire.

```
└─ [user@parrot] - [~/Desktop]
└─ $ cat texte.txt
Bonjour les chats
└─ [user@parrot] - [~/Desktop]
└─ $ cat
Pas d'output
```

Diagramme d'annotation :

- une ligne bleue relie `texte.txt` à l'étiquette "notre argument".
- une ligne rouge relie `cat` à l'étiquette "Notre programme".
- une ligne bleue relie `cat` à l'étiquette "argument vide".
- une ligne rouge relie l'absence de sortie à l'étiquette "Pas d'output".

Donc si on veut créer un programme qui prend des paramètres depuis son lancement (pas simplement avec une entrée utilisateur une fois le programme lancé), il faut le préciser dans les parenthèse de `main()`.

```

1 #include <stdio.h>$
2 $
3 void main(int argc, char *argv[]){$
4 $
5 }$

```

```

void main(int argc, char *argv[]){
<instruction>
}

```

void car ici nous ne « retournons » aucun résultat

main() notre fonction principale

int argc le nombre de paramètre dans la ligne de commande

char *argv[] un tableau de pointeur (notion approfondis plus tard) qui va regrouper nos arguments

{<instruction>} ici on va avoir nos instructions

Exemple pratique ;

```

1 #include <stdio.h>$
2 $
3 void main(int argc, char *argv[]){$
4     printf("%s\n", argv[1]);$
5     printf("%s\n", argv[2]);$
6 $
7 }$

```

Commentaire sur ce programme ;

l'objectif de ce programme est de print (= sortir en output dans le terminal) 2 mots.

Il faut noter qu'un tableau en C commence toujours par 0. Donc il existe bien un argv[0], mais pourtant on ne l'utilise pas, pourquoi ?

Simplement car la position 0 est le nom du programme lorsqu'on le lance dans le terminal. Dans cet exemple il s'agit de a.out (voir le lancement du programme).

Autre point à noter, normalement on prévoit des sécurités si l'utilisateur ne renseigne pas d'argument mais par simplicité pour l'exemple je n'ai rien fait.

```

$gcc test.c
[user@parrot]~[~/Desktop]
$./a.out les chats
les
chats
[x]-[user@parrot]~[~/Desktop]
$

```

Commentaire ; vous pouvez remarquer que lorsque vous n'utilisez pas d'options avec gcc le programme compilé est automatiquement nommé a.out

Les variables

Pour déclarer une variable, il faut faire attention au nom. On ne peut pas commencer par un nombre et on ne peut pas mettre d'espace.

Attention à ce que cela soit cohérent, un petit exemple avec la nomenclature (a jours ? attention à vous renseigner) ici dans l'école 42 ; <https://github.com/Nqsir/The-Norm-42>

- Un nom d'union doit commencer par `u_`.
- Un nom d'enum doit commencer par `e_`.
- Un nom de globale doit commencer par `g_`.
- Les noms de variables, de fonctions doivent être composés exclusivement de minuscules, de chiffres et de `'_'` (Unix Case).
- Les noms de fichiers et de répertoires doivent être composés exclusivement de minuscules, de chiffres et de `'_'` (Unix Case).
- Le fichier doit être compilable.
- Les caractères ne faisant pas partie de la table ascii standard ne sont pas autorisés.

Partie conseillée

- Les objets (variables, fonctions, macros, types, fichiers ou répertoires) doivent avoir les noms les plus explicites ou mnémoniques. Seul les 'compteurs' peuvent être nommés à votre guise.

Les variables peuvent être de différent types ;

-int ; integer (voir plus haut) / -32 768 à 32 767

-long ; integer mais avec plus de capacité / -2 147 483 648 à -2 147 483 647

-float ; nombre décimaux / 1.17549×10^{-38} à $3.40282 \times 10^{+38}$

-double ; nombre décimaux / 2.22507×10^{-308} à $1.79769 \times 10^{+308}$

-signed char ; char c'est pour le texte / -128 à 127

-unsigned char ; 0 à 255

-unsigned long ; 0 à 4 294 967 295

-unsigned int ; 0 à 65 535

-size_t ; il s'agit d'un unsigned int utilisé avec sizeof()

Il faut déclarer la variable pour l'utiliser ;

`<types_de_variable> <nom_de_la_variable>;`

Exemple ;

```
$ int main(){  
1   int chat;  
2   float personne;  
}
```

Pour affecter une valeur à une variable

`<nom_de_la_variable> = <valeur_de_la_valeur>;`

Exemple ;

```
chat = 2;  
personne = 7.3;
```

N'oubliez pas que « *l'ordinateur lit ligne par ligne* » (c'est un peu imagé mais c'est bien fait ligne par ligne). Dans notre exemple je pourrais changer la valeur chat en chat = 3 plus bas il va donc prendre cette valeur en note.

On peut faire la déclaration et l'affectation sur la même ligne.

Exemple cas 1 = cas 2 :

Cas 1 ;

```
int chat;$
float personne;$
chat = 2;$
personne = 7.3;$
```

Cas 2 ;

```
3 int main(){$
4     int chat = 2;$
5     float personne = 7.3;$
```

On arrive au même résultat mais nous fait gagner quelques lignes.

Il existe une possibilité de faire en sorte qu'une variable ne change jamais, c'est une **constante**.

Pour déclarer une constante ;

`const <type_de_variable> <nom_de_la_variable> = <valeur_de_la_variable> ;`

Ici il faut tout de suite assigné une valeur.

Printf() d'une variable ;

Si on veut print une valeur d'une variable dans un texte, comme printf() voici la structure ;

`printf(" ceci est un texte de %d mots", <nom_de_la_variable_>);`

%d c'est pour un int, voici un tableau avec plus de détails ;

%d	Int
%u	Unsigned int
%ld	Long
%f	Float
%lf	Double
%s	String
%c	Single character
%p	Pointer

Comment faire si on en a plusieurs ?

```
printf("il y a %d chats, pour une moyenne de %f milliers d'habitant\n", chat, personne);$
return 0;$
```

On fait la même chose mais on répète les variables après les virgules.

Récupérer une valeur de l'utilisateur ;

1. On doit déclarer la variable
2. On utilise scanf() qui vient aussi de stdio.h. Pour utiliser scanf()

```
scanf(« %d », &<nom_de_la_variable>);
```

On doit bien mettre le %d entre guillemet et mettre un & devant le nom de la variable

Exemple ;

```
3 int main(){$
4     int nombre_de_chat;$
5     printf("Combien vous avez de chat?\n");$
6     scanf("%d",&nombre_de_chat);$
7     printf("vous avez %d chats, ce n'est pas assez\n",nombre_de_chat);$
8     return 0;$
9 }
```

```
[user@parrot: ~/Desktop]
└─$ ./program
Combien vous avez de chat?
7
vous avez 7 chats, ce n'est pas assez
[user@parrot: ~/Desktop]
```

Les variables globales

Il faut savoir qu'une variable qui est dans une fonction, n'existe qu'au moment où la fonction est utilisée. Ensuite elle n'existe plus.

Si vous avez une fonction en dehors de main() (*allez voir la suite pour mieux comprendre ça*), vous ne pourrez pas l'utiliser une variable de cette fonction dans main() car elle n'existe plus.

Mais si vous voulez absolument avoir une variable en dehors de tout ça ; il suffit de la déclarer en dehors de la fonction et/ou de main().

```
Parrot Terminal
File Edit View Search Terminal Help
1 #include <stdio.h>$
2 $user@parrot:~/Desktop$
3 int a = 2;$am.c -o program
4 $user@parrot:~/Desktop$
5 int main(){$
6     int b = 3;$
7     int resultat = a * b;$
8     printf("a est %d\n",a);$
9     printf("le résultat de a fois b est %d\n",resultat);$
10    return 0;$
11 }$
~$user@parrot:~/Desktop$
program.c
~$user@parrot:~/Desktop$
└─$ ./program
a est 2
le résultat de a fois b est 6
```

J'ai bien déclaré la variable a en dehors de main(), je peux la récupérer maintenant dans main() mais aussi dans d'autres fonctions du projet (voir la partie sur les projets).

Si je ne veux avoir que cette variable dans ce fichier lors de la déclaration je fais ;
static <type> <nom_de_la_variable> = <valeur> ;

Opérations Mathématiques

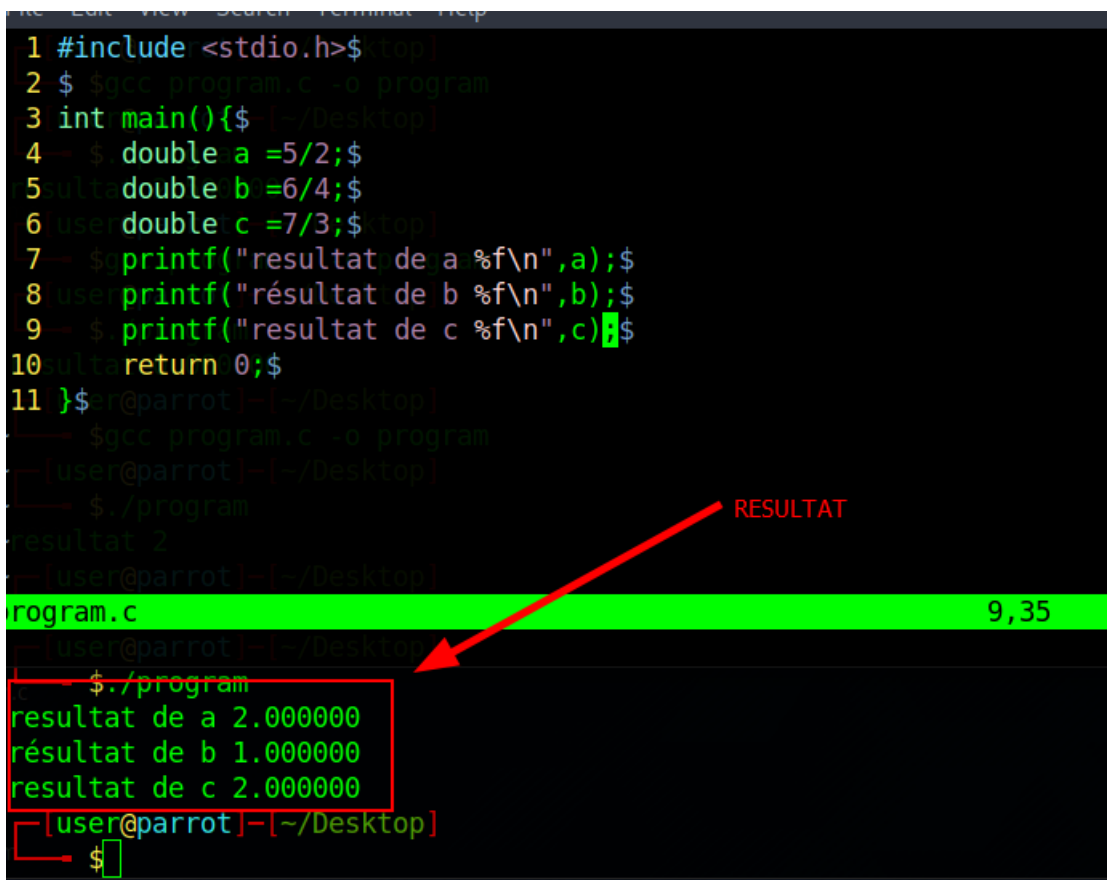
Les opérateurs mathématiques

Addition	+
Soustraction	-
Multiplication	*
Division	/
Modulo	%

Attention avec les division ;

Disclaimer=> si vous avez suivi le cours d'Openclasroom il y a une différence qui n'est pas précisé.

Si vous faites une division, il ne fait qu'une division euclidienne, c'est-à-dire sans chiffre après la virgule.



```
1 #include <stdio.h>$ top|
2 $ gcc program.c -o program
3 int main(){ $ ~/Desktop|
4 - $ double a =5/2;$
5 + double b =6/4;$
6 user double c =7/3;$ top|
7 - $ printf("resultat de a %f\n",a);$
8 user printf("résultat de b %f\n",b);$
9 - $ printf("resultat de c %f\n",c);$
10 + return 0;$
11 }$ @parrot| ~/Desktop|
    $ gcc program.c -o program
[user@parrot] ~/Desktop|
    $ ./program
resultat 2
[user@parrot] ~/Desktop|
program.c 9,35
[user@parrot] ~/Desktop|
    $ ./program
resultat de a 2.000000
résultat de b 1.000000
resultat de c 2.000000
[user@parrot] ~/Desktop|
    $
```

Quoi qu'il arrive on aura ça (même si on change `int main()` pour `double main()` ou `float...`) alors que j'ai bien mis du double en déclarant mes variables.

Cependant quelque chose qui n'est pas précisé dans le cours d'Openclassroom (en date de juillet 2024 et dernière mise à jours du cours fut le 14 février 2024), c'est que cela est différent si le calcul est fait d'une autre manière.

```
1 #include <stdio.h>
2 $ ./program
3 double main(){
4     double a =5;
5     double b =2;
6     double c =a/b;
7     printf("resultat de c %f\n",c);
8     return 0;
9 }
```

resultat de c 2.500000

program.c 6,17

"program.c" 9L, 140C written

```
$ gcc program.c -o program
[user@parrot]~$ ./program
resultat de c 2.500000
[user@parrot]~$
```

Si on passe le calcul dans les variables ça change tout et on finit bien avec un double et pas un résultat euclidien.

Autre cas, si vous passez deux int pour une variable double/float

```
1 #include <stdio.h>
2 $
3 int main(){
4     int a =5;
5     int b =2;
6     double c =a/b;
7     printf("resultat de c %f\n",c);
8     return 0;
9 }
```

Vous obtiendrez quand même 2.000000

Il faut passer des chiffres à virgules

Qu'est-ce que modulo ?

C'est le reste d'une division euclidienne.

5/2 => 2 en division euclidienne => il y a 2 fois 2 dans 5 => 2*2 =4 pour atteindre 5 il faut ajouter 1
5 = 2*2 +1
Donc 5%2 = 1

15/2 => 7 en division euclidienne => il y a 2 fois 7 dans 15 => 7*2 =14 pour atteindre 15 il faut ajouter 1
15 = 2*7 +1
Donc 15%2 = 1

Les raccourcis mathématiques

Très utile pour les boucles (voir plus bas), les raccourcis mathématiques permettent de faire des opérations d'incrémentation ou de décrémentation.

<variable>++; permet d'ajouter 1 à la valeur de la variable => qu'il faut déclarer + assigner une valeur en amont

<variable>--; permet de diminuer de 1 à la valeur de la variable => qu'il faut déclarer + assigner une valeur en amont

Pour les opérations ou vous voulez modifier votre variable de + - * / % x valeurs ;

<variable> <opérateur>= <valeur> ;

variable1 += 6 ; ajoute 6 à la variable1

variable1 -= 3 ; enlève 3 à la variable1

variable1 %=5 ; fait modulo 5 à la variable1

Pour faire plus de math, vous allez devoir puiser dans les librairies spécifiques.

Les conditions

Pour les conditions, il faut d'abord faire un test, voici les opérateurs ;

==	Egal
>	Supérieur
<	Inférieur
>=	Supérieur ou égal
<=	Inférieur ou égal
!=	Différent
&&	ET / AND
	OU / OR
!	NON / NO / NO (en espagnol)

Structure IF

```
if (<test>)  
{  
    <le_code_fait_des_choses>  
}  
else  
{  
    <le_code_fait_des_choses>  
}
```

La partie else n'est pas obligatoire c'est pour gérer les autres cas qui ne valident pas le premier test.

Exemple de code ;

```
Parrot Terminal
File Edit View Search Terminal Help
1 #include <stdio.h>$
2 $
3 int main(){$
4     int chats;$
5     printf("combien vous avez de chat?\n",chats);$
6     scanf("%d",&chats);$
7     if (chats == 0){$
8         printf("Pourquoi ?\n");}$
9     else{$
10        printf("pas mal, bon début\n");$
11    }$
12    return 0;$
13 }$
~$
~/Desktop
$ gcc program.c -o program
$ ./program
combien vous avez de chat?
0
Pourquoi ?
~/Desktop
$ ./program
combien vous avez de chat?
4
pas mal, bon début
~/Desktop
$
```

Oui les indentations sont pas terrible, fuck it

On peut faire aussi avec **else if** faire un second test. Voici la structure ;

```
If (<test>){
    <code>
}
else if(<second_test>){
    <code>
}
else{
    <code>
}
```

Exemple; avec un else if un peu spécial dans le test

```
File Edit View Search Terminal Help
1 #include <stdio.h>$
2 $
3 int main(){$
4     int chats;$
5     printf("combien vous avez de chat?\n",chats);$
6     scanf("%d",&chats);$
7     if (chats == 0){$
8         printf("Pourquoi ?\n");}$
9     else if(chats >= 1 && chats < 4){$
10        printf("pas mal, bon début\n");}$
11    else{$
12        printf("vous avez une bonne escouade");}$
13    return 0;$
14 }$
program.c
[user@parrot]~[~/Desktop]
$ ./program
combien vous avez de chat?
0
Pourquoi ?
[user@parrot]~[~/Desktop]
$ ./program
combien vous avez de chat?
1
pas mal, bon début
[user@parrot]~[~/Desktop]
$ ./program
combien vous avez de chat?
4
vous avez une bonne escouade [user@parrot]~[~/Desktop]
$
```

Oui j'ai pas mis \n à la fin du dernier printf(), fuck it

[Cas particulier du !](#)

! veut dire « n'est pas ». Il est différent du != qu'on a dans les tests.

On le met avant un test *if* (*!(<test>)*)

[Booléens](#)

1 = vrai / true

0 = faux / false

On peut attribuer un test à une variable => ce qui a pour conséquence de faire une valeur 1 ou 0 à la variable.

Il existe plein de manière de vérifier true or false. Une façon intéressante de faire est d'avoir une variable à qui on assigne un test d'une autre variable

```
3 int main(){  
4     int a = 2;  
5     int chats = a > 1;  
6     if (chats){  
7         printf("vrai");  
8     }  
9     else{  
10        printf("faux");  
11    }  
12    return 0;  
13 }
```

Ici le resultat va être vrai car $a > 1$, c'est peu intéressant dans cette exemple mais si on fait a avec une entrée d'utilisateur ça peut être sympa.

[SWITCH](#)

Quand vous avez plein de cas, vous n'avez pas forcément à faire une structure if qui pourrait être long. Imaginez 15 cas avec un if statement.....

Structure du switch ;

```
switch(<variable>)  
{  
    case <valeur_que_la_variable_peut_avoir> :  
        <instruction>;  
        break;  
    case <valeur_que_la_variable_peut_avoir> :  
        <instruction>;  
        break;  
    case <valeur_que_la_variable_peut_avoir> :  
        <instruction>;  
        break;  
    .....  
}
```

En gros l'idée c'est que la variable qu'on met dans switch() => chaque cas (case) prévoit une valeur de notre variable et du coup une instruction différente pour chaque cas.

Exemple venant de codewars ; site pour s'entraîner

```

switch (num) {
  case 1:
    return "Sunday";
  case 2:
    return "Monday";
  case 3:
    return "Tuesday";
  case 4:
    return "Wednesday";
  case 5:
    return "Thursday";
  case 6:
    return "Friday";
  case 7:
    return "Saturday";
  default:
    return "Wrong, please enter a number between 1 and 7";
}

```

default pour
les autres cas

ATTENTION il faut bien mettre break dans la plupart des cas pour éviter des erreurs. C'est parce qu'on utilise return qu'on utilise pas break dans ce cas.

[Le ternaire](#)

C'est une sorte de « one-liner » (comprendre une ligne de code) pour faire un statement if

Structure d'une expression ternaire ;

(<test>) ? <expression_si_vrai> : <expression_si_faux> ;

Les boucles

Boucle = on répète une suite d'instructions plusieurs fois.

[While](#)

Un des types de boucle est « while » == « tant que »

Structure du while ;

```

while (<condition>)
{
  <instructions>;
}

```

ATTENTION AVEC LES BOUCLES. Il faut éviter les boucles infini qui pourrait casser, votre code, votre VM, votre ordinateur, votre vie.....

Dans les conditions de la boucle *while*, on a souvent quelque chose comme « tant que la variable n'est pas égal/supérieur/inférieur à cette valeur » **MAIS** si dans votre code à la fin de votre boucle *while*, vous ne changez pas la valeur de votre variable => **CATASTROPHE**

En gros si je devais écrire ce qui se passe ligne par ligne en utilisant notre langage ;

-valeur de ma variable = 0

-début de ma boucle while

-tant que ma variable n'est pas = 10 je continue à printf(« chats »). On appelle aussi la variable un compteur

-fin de mon instruction, je retourne au début de ma boucle while

-tant que ma variable n'est pas = 10 je continue à printf(« chats ») **effectivement ma variable est toujours à 0 en dehors de ma boucle je n'ai pas fait de modification.**

-je printf(« chats »)

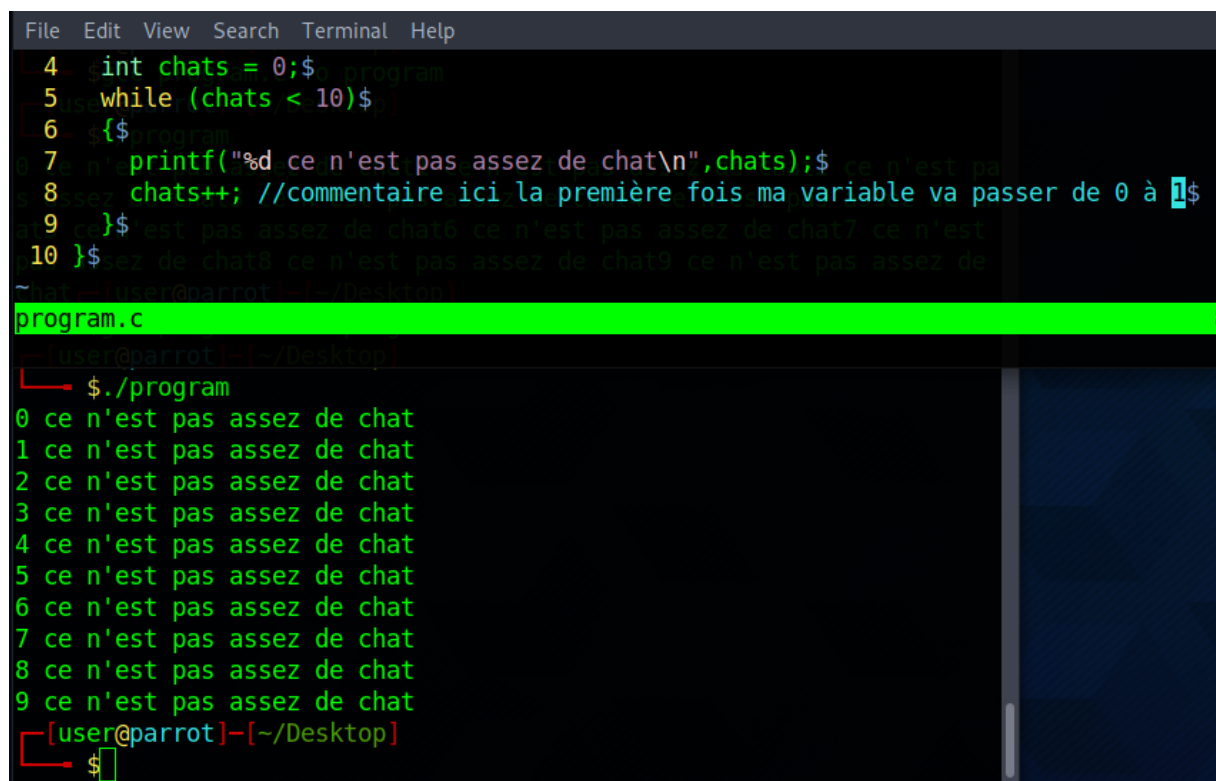
-fin de mon instruction, je retourne au début de ma boucle while

-tant que ma variable n'est pas = 10 je continue à printf(« chats ») **effectivement ma variable est toujours à 0 en dehors de ma boucle je n'ai pas fait de modification.**

-etc.... etc.....

C'est ici qu'on utilise les raccourcis qu'on a appris pour incrémenter une valeur.

Exemple ;



```
File Edit View Search Terminal Help
4 int chats = 0;$ ./program
5 while (chats < 10){
6 {
7 printf("%d ce n'est pas assez de chat\n",chats);$
8 chats++; //commentaire ici la première fois ma variable va passer de 0 à 1$
9 }$
10 }$
~chat [user@parrot] ~/Desktop
program.c
[user@parrot] ~/Desktop
$ ./program
0 ce n'est pas assez de chat
1 ce n'est pas assez de chat
2 ce n'est pas assez de chat
3 ce n'est pas assez de chat
4 ce n'est pas assez de chat
5 ce n'est pas assez de chat
6 ce n'est pas assez de chat
7 ce n'est pas assez de chat
8 ce n'est pas assez de chat
9 ce n'est pas assez de chat
[user@parrot] ~/Desktop
$
```

Ici tout va bien, à noter ici quelques petits points ;

-ca commence forcément à 0

-avec mon test < 10 ca exclut donc 10 dans mon printf()

-mon code parle de chat

Second exemple ; mauvais code

```
File Edit View Search Terminal Help
1 #include <stdio.h>$
2 $
3 int main(){$
4     int chats = 0;$ 'est pas assez de chat
5     while (chats != 10)$ pas assez de chat
6     {$ $ Home 7 ce n'est pas assez de chat
7         printf("%d ce n'est pas assez de chat\n",chats);$
8     }$ 9 ce n'est pas assez de chat
9 $ (user@parrot)~| ~/Desktop|
~ Trash $gcc program.c -o program
```

Ici pas bon ! A noter que j'ai fait un changement sur mon test qui est une autre façon de faire avec != mais qui arrive au même résultat excluant 10 dans mon printf().
Le plus gros problème ici, c'est qu'il ne va jamais finir la condition ! La variable ne va jamais atteindre 10.

[Do...while](#)

Dans une boucle *while* on fait un test puis on fait des instructions. Pour répéter tout ça car on a augmenté la valeur de notre variable dans le test.

Ici dans cette structure on va faire l'inverse. On va d'abord faire les instructions et ensuite le test avec *while*. **Attention**, on incrémente encore une fois bien notre variable (qu'on appelle aussi compteur) dans les instructions.

Voici la structure du do while

```
do
{
    <instructions>;
    <incrimente_mon_compteur>;
} while (<test>;
```

Cette structure permet d'exécuter au moins une fois les instructions. Ça peut être un objectif très précis et même rare mais au moins c'est arriver une fois. N'oubliez pas le programme va être « lu » par l'ordinateur ligne par ligne.

[For](#)

Structure de la boucle for ;

```
for (<variable> = <valeur>; <condition>; <incrementation_de_notre_variable>)
{
    <instructions>;
}
```

- Ici on assigne une valeur à notre variable dans le for statement
- Mais il faut bien déclarer notre variable avant de commencer notre for
- <condition> est identique que dans while, on utilise < == > != etc...
- ici au moins on ne peut pas se tromper pour une boucle infini car l'incrémentation doit être faite tout de suite.

```

1 #include <stdio.h>$
2 $
3 int main(){$
4     int chats = 0;$
5 $
6     for(chats=0;chats<10;chats++){
7         printf("il n'y a que %d chats\n",chats);$
8     }$
9 $
10 }$

```

Les fonctions

Plus de détails sur l'écriture de fonction. Pour le moment on a toujours fait nos opérations dans main() qui est la fonction principale du programme.

Mais dans main(), vous allez faire référence à d'autres fonctions que vous allez écrire en dehors de main().

Voici la structure des fonctions ;

```

<type> <function_name>(<parameters>)
{
    instructions ;
}

```

<type> c'est le type de données qu'on va avoir en sortie, en résultat, de notre fonction. Cela peut être du int, double, float....Mais aussi void si elle ne renvoie rien.

<function_name>(<parameters>) c'est le nom de la fonction, puis des paramètres si cela est nécessaire.

Dans les paramètres, on met encore une fois le <type> <nom_du_paramètre>

L'ordre

On a dit au tout début de ce document, que le programme commence « toujours » par main().

Le « toujours » est entre guillemet car c'est plus compliqué que ça. Le programme va effectivement utiliser main() en premier. Mais si dans main() vous faites référence à une fonction qui est en dehors, l'ordinateur qui lit ligne par ligne le code va vous dire ; « mais de quoi tu parles mec ? moi j'ai jamais entendu parler de ça ? t'es ouf ! ».

Exemple de ce problème, expliqué ligne par ligne ;

```

1 #include <stdio.h>$
2 $
3 $
4 int main(){$
5     int a = 2;$
6     int b = 3;$
7     int resultat = multiplication(a,b);$
8     printf("%d\n",resultat);$
9     return 0;$
10 }$
11 $
12 $
13 $
14 int multiplication(int a, int b){$
15     return a * b;$
16 }$

```

L'ordinateur fait ;

-je lis la ligne 1 : ok

-je lis la ligne 4 : ok

-ah ! ligne 4 c'est mon programme à faire, je me mets au travail : ok

-ligne 5 + ligne 6 j'ai 2 variables : ok

-ligne 7 je déclare une variable...ok....ok....elle est égale à.....multiplication(a,b).....Mais c'est quoi ca ? **JE N'AI RIEN LU A CE SUJET AAAHHHH DU COUP JE TRAVAIL PLUS VOILA erreur de compilation**

```

[user@parrot]~/Desktop
$ gcc program.c -o program
program.c: In function 'main':
program.c:7:20: warning: implicit declaration of function 'multiplication' [-Wimplicit-function-declaration]
7 |     int resultat = multiplication(a,b);
  |                    ^

```

Pour pallier à ce problème, on va faire un *prototype*. Après nos #include des librairies, on va mettre la première ligne de notre fonction,

```

1 #include <stdio.h>$
2 $
3 int multiplication(int a, int b);$
4 $
5 int main(){$
6     int a = 2;$
7     int b = 3;$
8     int resultat = multiplication(a,b);$
9     printf("%d\n",resultat);$
10    return 0;$
11 }$
12 $
13 $
14 $
15 int multiplication(int a, int b){$
16     return a * b;$
17 }$

```

La c'est bon

Si on recommence par lire, ligne par ligne ;

-je lis la ligne 1 : ok

-je lis la ligne 3, oh ! c'est une fonction pour plus tard d'accord.

-je lis la ligne 5, c'est mon programme cool

-ligne 6 et 7, des variables : ok

-ligne 8, une variable qui est le résultat de la fonction multiplication(a,b).....ah ! je connais cette fonction ! C'est bon je vais la retrouver !

A noter on a mis un ; sur la ligne 3 c'est ce qui fait comprendre à la machine que c'est pas la déclaration de la fonction mais juste un prototype.

Les projets

Un logiciel n'est pas simplement un fichier qu'on a compilé et qu'on exécute, c'est plus souvent des projets avec plusieurs fichiers.

Dans votre projet, [vous allez avoir des fichiers sources en .c et des headers en .h](#)

Les fichiers en .h c'est les prototypes des fonctions alors que les fichiers en .c c'est le code des fonctions.

Dans votre fichier en .c, vous allez le prévenir qu'il y a les prototypes, mais pas de la manière dont on fait ! Dans la précédente partie on a fait ça car c'est un petit programme mais imaginez bien des programmes avec des milliers de lignes !

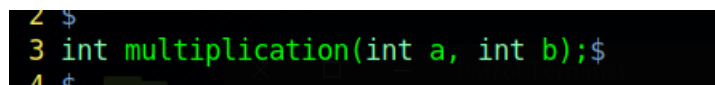
On va le mettre à la suite des #include



```
1 #include <stdio.h>;
2 #include "multiplication.h";
3
4
5 int main(){
```

Ici dans le #include on met cette fois entre guillemet « »

Dans votre fichier en .h c'est là ou vous allez retrouver les prototypes comme on a fait au préalable ;



```
2
3 int multiplication(int a, int b);
4
```

[GCC](#)

On a vu gcc pour compiler notre code pour le rendre exécutable. Mais on peut le faire aussi sur des projets entiers avec différents fichiers.

Exemple ;

```

[user@parrot]--[~/Desktop/project]
$ pwd
/home/user/Desktop/project
[user@parrot]--[~/Desktop/project]
$ ls
main.c multiplication_division.c multiplication_division.h
[user@parrot]--[~/Desktop/project]
$

```

pwd => print working directory => c'est à dire où je suis dans ma machine linux

ls => listing => montre tout ce qu'il y a de visible dans ce fichier

On retrouve donc dans mon dossier « project », 3 fichiers (main.c, multiplication_division.c et multiplication_division.h).

Les fichiers en .c c'est là où il y a le code alors que le .h c'est le header.

Mon main.c ;

```

1 #include <stdio.h>$
2 #include "multiplication_division.h"$
3 $
4 int main(){$
5     int a, b;$
6     printf("le premier chiffre\n");$
7     scanf("%d",&a);$
8     printf("le second chiffre\n");$
9     scanf("%d",&b);$
10    multiplication_division(a,b);$
11    return 0;$
12 }$

```

On peut voir en ligne 2 que je fais appel au header multiplication_division.h entre guillemet.

Sur la ligne 10, j'ai une fonction multiplication_division() qui n'est pas dans main.c =>

l'ordinateur va la retrouver grâce au header. Donc ici je n'ai pas fait de prototype comme on a pu voir avant.

Dans le fichier multiplication_division.c ;

```

1 #include <stdio.h>$
2 #include "multiplication_division.h" //on include aussi ici notre .h$
3 $
4 $
5 int multiplication_division(int a, int b){$
6     int multiplication = a*b;$
7     int division = a/b;$
8     printf("le resultat de la multiplication est %d, le resultat de la division est %d\n",multiplication,division);$
9     return 0;$
10 }$

```

Simplement, il nous fait une multiplication et une division pour finir par un printf().

Enfin, multiplication_division.h ; (le premier include est pas obligatoire)

```
1 #include <stdio.h>
2 int multiplication_division(int a, int b);
```

J'ai mon prototype de ma fonction.

Maintenant avec gcc on va tout compiler pour mettre ca ensemble et l'exécuter.

```
[user@parrot]~/Desktop/project
$gcc main.c multiplication_division.c multiplication_division.h -o project
[user@parrot]~/Desktop/project
$ls
main.c multiplication_division.c multiplication_division.h project
[user@parrot]~/Desktop/project
$./project
le premier chiffre
2
le second chiffre
3
le resultat de la multiplication est 6, le resultat de la division est 0
[user@parrot]~/Desktop/project
$
```

Description ;

-ligne 1 = la commande qui me permet de compiler l'ensemble.

gcc <fichier1.c> <fichier2.c>.....<fichier_headers>.... -o <nom_que_je_veux>

rappel -o c'est pour output.

-ligne 2 = ls pour listing et voir si j'ai bien mon « project » qui est créer mais comme je n'ai pas fait d'erreur (sinon bloque à la compilation) tout est bon.

-ligne 3 = je lance mon programme tout fonctionne bien. **Ce qu'il faut comprendre** => je n'ai pas fait de prototype dans main.c => il vient directement de multiplication_division.h que j'ai include => je pourrais faire ca avec plus de fichiers (des dizaines, des centaines....) => bien sur ici c'est un peu léger c'est pour l'exemple mais imaginez sur des milliers de ligne de code.

Après le beau temps, vient la pluie.

Imaginons, que je dois ajouter quelque chose dans multiplication_division.c (une fonctionnalité, une phrase...). Si je modifie ce fichier, je vais devoir TOUT recompiler. Refaire la commande gcc avec tout dedans. Ce qui n'est pas très grave dans notre cas car c'est un petit projet mais imaginez encore une fois sur des milliers de ligne de code => cela peut être long.

Surtout que le reste je n'y touche pas, c'est juste imaginons une faute de frappe quelque part.

Après la pluie, le beau temps

Pour ça il y a une autre manière de faire avec make et makefile

[Make, mafile](#)

Pour que l'outil make fonctionne, il nous faut un « makefile »

```

program: program.o weatherstats.o
    gcc -std=c11 -Wall -fmax-errors=10 -Wextra program.o weatherstats.o -o program

program.o: program.c weatherstats.h
    gcc -std=c11 -Wall -fmax-errors=10 -Wextra -c program.c -o program.o

weatherstats.o: weatherstats.c
    gcc -std=c11 -Wall -fmax-errors=10 -Wextra -c weatherstats.c -o weatherstats.o

clean:
    rm -f weatherstats.o program.o program

```

Structure d'une commande

<target> : **<component1>** **<component2>**
<gcc_command>

<target> c'est ce qu'on cherche à compiler. Vous pouvez voir dans certaines lignes des fichiers qui finissent par .o

Il s'agit d' **object files**, il s'agit d'une étape intermédiaire dans la compilation que l'on peut faire avec la commande gcc. Jusqu'à présent on n'en avez pas besoin car on fait des petits programmes.

<component1> **<component2>** on retrouve ici ce qui est nécessaire comme fichier pour créer notre target. Il peut y avoir plusieurs fichiers comme un seul.

<gcc command> ici on a des options en plus qui servent à avoir plus d'informations s'il y a un problème.

On affiche au maximum 10 erreurs avec fmax

Wall va nous donner des avertissements + des variables non utilisées

.....

Donc quand on va faire la commande **make program** pour la première fois il va faire plusieurs opérations ;

-compiler program.o

-compiler weatherstats.o

-compiler program grâce aux deux autres

Si on se rend compte qu'on a besoin de faire une modification dans program, on va refaire **make program**. Mais imaginez qu'il ne s'agit que d'une modification qui ne concerne qu'un printf() qui n'est pas en lien avec weatherstats.

Il ne va pas faire d'update de weatherstats ! Il sait que ce ne le concerne pas et il a déjà le weatherstats.o nécessaire.

C'est la force de make / makefile il va faire la compilation la ou il y a des choses à faire et pas des choses inutiles.

Nous sommes dans un cas simple et un peu bête mais globalement ca peut déjà faire gagner du temps.

make clean est une commande qui va supprimer nos fichiers (ici dans cette exemple c'est pas super malin de supprimer ca mais au moins on est tranquille).

[Makefile ressources](#)

<https://makefiletutorial.com/>

[vidéo youtube sur ma chaine incoming en janvier 2025]

Les pointeurs

Vidéo youtube sur les pointeurs ; <https://www.youtube.com/watch?v=MmZyGL5s290>

La définition est la suivante ; « **un pointeur pointe vers l'adresse dans la mémoire de l'ordinateur d'une variable** ».

Revenons déjà sur les variables.

Quand on assigne une valeur à une variable ; **int chats = 10** ; l'ordinateur va garder en mémoire cette information. L'information à une adresse dans l'ordinateur ! Il ne va pas conservé l'information « chats » mais une adresse.

Quand je veux retrouver l'information, j'utilise le nom de la variable **chats**. Mais l'ordinateur lui en réalité. Il va chercher à l'adresse qu'il a enregistré l'information.

On peut déjà sans utiliser de pointeur, chercher l'adresse de cette variable. Pour ça il suffit de mettre un & devant le nom de la variable.

Il faut aussi faire une modification dans printf(), on va utiliser %p.

```
1 #include <stdio.h>$
2 $
3 $
4 int main(){$
5     int chats = 10;$
6     printf("la valeur de la variable chats est de %d\n",chats);$
7     printf("l'adresse de la variable chats est la suivante %p\n",&chats);$
8 }$
9 $
10 $
```



```
$ ./program
la valeur de la variable chats est de 10
l'adresse de la variable chats est la suivante 0x7ffd2e077dec
```

Maintenant, on peut regarder les pointeurs. Le pointeur va pointer la direction d'une adresse dans la mémoire.

Dans notre cas, on va pointer l'adresse de la variable chats.

Comment déclarer un pointeur ?

int *<nom_du_pointeur> ;

Ici c'est la création simple. Comme pour une variable on indique un type. La seule différence c'est le fait qu'on mette une * devant le nom de notre pointeur.

int *<nom_du_pointeur> = NULL ;

En ajoutant = NULL, on crée notre pointeur mais on lui indique de nous réserver une case vide de mémoire dans la l'ordinateur.

int *<nom_du_pointeur> = &<nom_de_la_variable>;

Enfin ici on retrouve ce qu'on vient de faire &<nom_de_la_variable> pour lui dire que c'est l'adresse de cette variable qui nous intéresse.

```
1 #include <stdio.h>$
2 $
3 $
4 int main(){$
5     int chats = 10;$
6     printf("la valeur de la variable chats est de %d\n",chats);$
7     printf("l'adresse de la variable chats est la suivante %p\n",&chats);$
8 $
9     int *pointeur = &chats;$
10    printf("l'adresse qui est enregistré dans notre pointeur est la suivante %p\n", pointeur);$
11 $
12 }$
```

Ligne 9, je lui demande d'enregistrer l'adresse de la variable chats.

En résultat ;

```
[user@parrot]-(~/Desktop)
└─$ ./program
la valeur de la variable chats est de 10
l'adresse de la variable chats est la suivante 0x7ffde7eb0444
l'adresse qui est enregistré dans notre pointeur est la suivante 0x7ffde7eb0444
[user@parrot]-(~/Desktop)
```

On voit bien que l'adresse (qui est exprimé en hexadecimal) est identique. SUCESS !

Maintenant si je souhaite interroger mon pointeur, sur la valeur qui est dans cette adresse ?

Dans mon printf(), je dois mettre %d de nouveau car il s'agit d'un int et mettre *<nom_du_pointeur>

```
1 #include <stdio.h>$
2 $
3 $
4 int main(){$
5     int chats = 10;$
6     printf("la valeur de la variable chats est de %d\n",chats);$
7     printf("l'adresse de la variable chats est la suivante %p\n",&chats);$
8 $
9     int *pointeur = &chats;$
10    printf("l'adresse qui est enregistré dans notre pointeur est la suivante %p\n", pointeur);$
11    printf("la valeur qui est enregistré dans pointeurs est la même que chats %d\n", *pointeur);$
12 }$
13 $ la suivante 0x7ffde7eb0444
14 $
```

```
[user@parrot]-(~/Desktop)
└─$ ./program
la valeur de la variable chats est de 10
l'adresse de la variable chats est la suivante 0x7ffdb1116ba4
l'adresse qui est enregistré dans notre pointeur est la suivante 0x7ffdb1116ba4
la valeur qui est enregistré dans pointeurs est la même que chats 10
[user@parrot]-(~/Desktop)
└─$
```

Il faut bien comprendre que les deux sont en lien !

Si je fais une modification sur la variable chats ;

```

1 #include <stdio.h>$
2 $
3 $
4 int main(){$
5     int chats = 10;$
6     printf("la valeur de la variable chats est de %d\n",chats);$
7     printf("l'adresse de la variable chats est la suivante %p\n",&chats);$
8     $ la suivante 0x7ffc773ff61d
9     int *pointeur = &chats;$
10    printf("l'adresse qui est enregistré dans notre pointeur est la suivante %p\n", pointeur);$
11    printf("la valeur qui est enregistré dans pointeur est la même que chats %d\n", *pointeur);$
12 $
13    //maintenant je vais une opération sur la variable chats$
14    chats +=2;$
15    printf("la valeur de la variable chats est de %d\n",chats);$
16    printf("la valeur qui est enregistré dans pointeur est la même que chats %d\n", *pointeur);$
17 }$ 0x700f354

```

```

$ ./program
la valeur de la variable chats est de 10
l'adresse de la variable chats est la suivante 0x7ffd9799f354
l'adresse qui est enregistré dans notre pointeur est la suivante 0x7ffd9799f354
la valeur qui est enregistré dans pointeur est la même que chats 10
la valeur de la variable chats est de 12
la valeur qui est enregistré dans pointeur est la même que chats 12

```

J'ai modifié la valeur de la variable chats. Du coup logiquement, comme pointeur n'est qu'une redirection. J'ai retrouvé la même chose

Je peux aussi manipulé la variable chats, à partir du pointeur. Ils sont liées ensemble donc si je modifie l'un ca modifie l'autre.

```

1 #include <stdio.h>$
2 $
3 $
4 int main(){$
5     int chats = 10;$
6     printf("la valeur de la variable chats est de %d\n",chats);$
7     printf("l'adresse de la variable chats est la suivante %p\n",&chats);$
8     $ la suivante 0x7ffde9274b4
9     int *pointeur = &chats;$
10    printf("l'adresse qui est enregistré dans notre pointeur est la suivante %p\n", pointeur);$
11    printf("la valeur qui est enregistré dans pointeur est la même que chats %d\n", *pointeur);$
12 $
13    //maintenant je vais une opération sur le pointeur$
14    *pointeur+=2; //attention ici je fais *pointeur$
15    printf("la valeur de la variable chats est de %d\n",chats);$
16    printf("la valeur qui est enregistré dans pointeur est la même que chats %d\n", *pointeur);$
17 }$ 72ade944

```

ATTENTION je fais *pointeur

```

$ ./program
la valeur de la variable chats est de 10
l'adresse de la variable chats est la suivante 0x7ffd72ade944
l'adresse qui est enregistré dans notre pointeur est la suivante 0x7ffd72ade944
la valeur qui est enregistré dans pointeur est la même que chats 10
la valeur de la variable chats est de 12
la valeur qui est enregistré dans pointeur est la même que chats 12

```

J'ai de nouveau la même valeur.

Vous allez souvent entendre parler de «*déréférencement*» «*dereferencing*» «*pointers* *dererence*».... C'est le terme qu'on utilise quand on utilise le sigle * pour accéder à la valeur

de l'adresse pointée. Cela peut prendre de l'importance dans le fonctionnement des tableaux et des chaînes de caractères.

Si on revient en arrière dans le cours, on a parlé de `scanf()`, on avait déjà utilisé `&` sans savoir qu'il s'agit d'une adresse de la variable ;

```
Parrot Terminal
File Edit View Search Terminal Help

1 #include <stdio.h>$
2 $
3 $
4 int main(){$
5 +++++$
6 int a = 0;$
7 int b = 0;$
8 printf("premier chiffre\n");$
9 scanf("%d",&a);$
10 printf("second chiffre\n");$
11 scanf("%d",&b);$
12 printf(" a = %d, et b = %d\n",a,b);$
13 }$

program.c
[user@parrot]~[~/Desktop]
$ gcc program.c -o program
[user@parrot]~[~/Desktop]
$ ./program

premier chiffre
8
second chiffre
4
a = 8, et b = 4
[user@parrot]~[~/Desktop]
$
```

Si on examine ce code ;

```

1 #include <stdio.h>$
2 $
3 int addition(int a, int b);$
4 $
5 int main(){$
6 ++++$
7     int a = 0;$
8     int b = 0;$
9     printf("premier chiffre\n");$
10    scanf("%d",&a);$
11    printf("second chiffre\n");$
12    scanf("%d",&b);$
13    printf(" a = %d, et b = %d\n",a,b);$
14    addition(a,b);$
15    int resultat;$
16    printf("le resultat de a + b est  %d\n",resultat);$
17 }$
18 $
19 int addition(int a, int b){$
20     int resultat = a+b;$
21     return 0;$
22 }$

```

On peut voir qu'il ne va pas fonctionner comme je veux ;

```

$ ./program
premier chiffre
8
second chiffre
2
 a = 8, et b = 2
le resultat de a + b est  0

```

Pourquoi ? Parce que la variable résultat dans la fonction main() et dans la fonction addition() ne sont pas les mêmes. Celle de la fonction addition(), n'existe que pendant l'exécution de celle-ci. On peut le « prouver » avec un printf() dans la fonction addition().

```

1 #include <stdio.h>$
2 $
3 int addition(int a, int b);$
4 $
5 int main(){
6 +++$
7     int a = 0;$
8     int b = 0;$
9     printf("premier chiffre\n");$
10    scanf("%d",&a);$
11    printf("second chiffre\n");$
12    scanf("%d",&b);$
13    printf(" a = %d, et b = %d\n",a,b);$
14    addition(a,b);$
15    int resultat;$
16    printf("le resultat de a + b est  %d\n",resultat);$
17 }$
18 $
19 int addition(int a, int b){
20     int resultat = a+b;$
21     printf("la variable resultat de la fonction addition est égale à %d\n",resultat);$
22     return 0;$
23 }$

```



```

[user@parrot]~[~/Desktop]
$ ./program
premier chiffre
8
second chiffre
2
a = 8, et b = 2
la variable resultat de la fonction addition est égale à 10
le resultat de a + b est 0

```

Moi je souhaite trouver résultat dans main(). Je pourrais bien sur la créer dans main() mais mon objectif c'est de l'avoir avec un pointeur.

Pour faire ça, il faut changer la fonction.

Il faut lui indiquer qu'on va prendre un paramètre en plus.

```

1 #include <stdio.h>$
2 $
3 int addition(int a, int b, int *resultat);$
4 $
5 int main(){
6 +++$
7     int a = 0;$
8     int b = 0;$
9     printf("premier chiffre\n");$
10    scanf("%d",&a);$
11    printf("second chiffre\n");$
12    scanf("%d",&b);$
13    printf(" a = %d, et b = %d\n",a,b);$
14    int resultat; //changement de place pour le mettre avant la fonction addition$
15    addition(a,b,&resultat); //ici j'ai mis & pour indiquer que je met l'adresse de la variable resultat dans main()$
16    printf("le resultat de a + b est  %d\n",resultat);$
17 }$
18 $
19 int addition(int a, int b,int *resultat){
20     *resultat = a+b; //ici j'ai mis de nouveau * pour le pointeur resultat$
21     return 0;$
22 }$

```

J'ai mis quelques commentaires pour montrer ce que j'ai changé.

En gros l'idée est la suivante ; suivre nos premiers exemple de modification à partir du pointeur.
Ma fonction addition(), prend 3 paramètres ;

-a

-b

-*resultat

Qu'est-ce que je dis à l'ordinateur ? Voici ma conversation avec l'ordinateur :

-Monsieur l'ordinateur, j'ai une fonction qui prend 3 paramètres

-ok

-les 2 premiers sont des int

-ok

-Le troisième est une **ADRESSE d'une variable**. Variable que j'ai déjà déclarer (int resultat), je vous le signale par le symbole &

-ok.....donc en gros ***pointeur = &resultat**. Ici le pointeur vous l'avez aussi appeler « resultat ». En soit c'est comme si vous faisiez cette ligne la pour déclarer un pointeur.

-c'est ca monsieur l'ordinateur !

-donc si vous faites « *resultat = a + b » comme c'est lié à la variable resultat dans main() ca va modifier aussi la valeur. Comme vous avez fait dans l'exemple plus haut avec une modification du pointeur (attention à bien mettre *).

-Exactement, je vois qu'on se comprend, merci beaucoup bonne journée !

-Bonne journée

On va voir un second cas, qui va nous permettre de faire quelque chose qui jusqu'à présent était impossible !

A partir d'une fonction, avoir 2 résultats ! Jusqu'à présent, on ne pouvait avoir qu'un seul résultat dans le return. Mais avec ce qu'on a découvert pour les pointeurs dans les fonctions, on va pouvoir faire le même types de liens que dans l'exemple précédent.

```
1 #include <stdio.h>$
2 $ [~Wint-conversion]
3 int multiplication_division(int a, int b, int *pointeur1, int *pointeur2);$
4 $
5 int main(){[user@parrot]~/Desktop}
6 ++++$ gcc program.c -o program
7 int a = 0;$[parrot]~/Desktop}
8 int b = 0;$ program
9 int multiplication, division;$
10 printf("premier chiffre\n");$
11 scanf("%d",&a);$
12 printf("second chiffre\n");$
13 scanf("%d",&b);$
14 printf(" a = %d, et b = %d\n",a,b);$
15 multiplication_division(a,b,&multiplication,&division);$
16 printf("le resultat de la multiplication est %d\n",multiplication);$
17 printf("le resultat de la division est %d\n",division);$
18 }$
19 $ premier chiffre
20 int multiplication_division(int a, int b,int *pointeur1, int *pointeur2){$
21 *pointeur1 = a * b;$
22 *pointeur2 = a / b;$
23 return 0;$ et b = 2
24 }$ le resultat de la multiplication est 16
```

Bref explication (je vous laisse bien décortiqué), mais dans l'idée j'ai envoyé des adresses de variable dans une fonction. La fonction est bien déclaré avec des pointeurs (comme on a pu voir avec int *<nom_du_pointeur>), ce qui correspond à écrire dans l'idée (c'est pas tout a fait ca attention) *pointeur1 = &multiplication. Du moins c'est ce que comprend la machine

```

$ ./program
premier chiffre
8
second chiffre
2
a = 8, et b = 2
le resultat de la multiplication est 16
le resultat de la division est 4

```

Les tableaux / array

Les tableaux sont une suite de variable de même types. On appelle cela aussi un array.

Il faut comprendre que ce tableau est un pointeur lui aussi. Si vous cherchez son adresse, vous allez avoir l'adresse du premier élément qu'il indexe. La suite des éléments étant à une adresse suivante.

Comment déclarer un tableau ?

<types> **<nom_tableau>** [**<nb_valeur>**];

<types> comme quand on déclare une variable int / double / long /.....

<nb valeur> on doit lors de la déclaration de notre tableau, lui donner l'information en amont de sa longueur. Car l'ordinateur va réserver des places dans sa mémoire pour le tableau.

Ensuite pour assigner des valeurs dans notre tableau ;

<nom_tableau> [**<espace>**] = **<valeur>** ;

<espace> c'est « le nombre de case » de mémoire qu'on veut remplir. Attention on commence comme d'habitude par 0.

```

1 #include <stdio.h>$
2 $
3 $
4 int main(){$
5     int tableau[4];$
6     tableau[0] = 1;$
7     tableau[1] = 2;$
8     tableau[2] = 3;$
9     tableau[3] = 4;$
10    return 0;$
11 }$
12 $

```

On peut voir l'adresse du tableau, qui correspond en réalité à l'adresse de la première case. ;

```
Parrot Terminal
File Edit View Search Terminal Help
1 #include <stdio.h>$
2 $
3 $ | return 0;$
4 int main(){$
5 |-[uint@tableau[4];$desktop]
6 |-$gcc tableau[0]=1;$program
7 |-$gcc tableau[1]=2;$top]
8 |-$./tableau[2]=3;$
9 |-$gcc tableau[3]=4;$[user@parrot]~[~/Desktop]
10 |-$gcc program.c -o program
11 |-$gcc printf("l'adresse du tableau %p\n",tableau);$
12 |-$./return 0;$
13 |-$leau 0x7ffc764c97d0, [user@parrot]~[~/Desktop]
program.c
[user@parrot]~[~/Desktop]
$ ./program
l'adresse du tableau 0x7fff02028b90
```

On peut aussi printf() les valeurs du tableau

```
Parrot Terminal
File Edit View Search Terminal Help
1 #include <stdio.h>$
2 $
3 $ | View Search Terminal Help
4 int main(){$
5 | int tableau[4];$
6 | tableau[0] =1;$
7 |-[uint@tableau[1] =2;$desktop]
8 |-$gcc tableau[2] =3;$program
9 |-$gcc tableau[3] =4;$top]
10 |-$./program
11 |-$gcc printf("la valeur du premier élément du tableau %d\n",*tableau);$
12 |-$gcc printf("la troisième valeur du tableau %d\n", tableau[2])$
13 |-$return 0;$ Desktop]
14 |-$ ./program
15 |-$leau 0x7ffc764c97d0, [user@parrot]~[~/Desktop]
16 |-$gcc program.c -o program
17 |-$r@parrot]~[~/Desktop]
18 |-$./program
~ adresse du tableau 0x7fff02028b90
~ [user@parrot]~[~/Desktop]
program.c
"program.c" 18L, 284C written
$ ./program
la valeur du premier élément du tableau 1
la troisième valeur du tableau 3
```

On peut aussi initialiser un tableau d'une manière différente ;

Dans ce cas toutes les valeurs sont à 0

int <tableau>[<espace>] = {1} Attention lorsqu'il ne s'agit pas de 0, vous allez obtenir 1 en première valeur et 0 aux autres ;

On peut donc créer un tableau de cette manière

```
Int <tableau>[] = {<Valeur>,<Valeur>,...,<valeur>}
```

Rappel ; je mets int car je veux un tableau avec des int, il est possible de mettre d'autres types en fonction de ce qu'on veut faire.

```
Parrot Terminal
File Edit View Search Terminal Help
1 #include <stdio.h>$tableau 0
2 $troisième valeur du tableau -1092566576
3 $r@parrot|-[~/Desktop]
4 int main(){$m.c -o program
5 user@parrot:~$ gcc -o program
6 $ ./program
7 prem printf("la première valeur du tableau %d\n", tableau[0]);$
8 seco printf("la seconde valeur du tableau %d\n", tableau[1]);$
9 troi printf("la troisième valeur du tableau %d\n", tableau[2]);$
10 quat printf("la quatrième valeur du tableau %d\n", tableau[3]);$
11 }$r@parrot|-[~/Desktop]
program.c [+]
"program.c" 15L, 328C written
$ ./program
la première valeur du tableau 101
la seconde valeur du tableau 45
la troisième valeur du tableau 8
la quatrième valeur du tableau 666
```

AU tout début de cette section sur les tableaux / array. J'ai précisé qu'il s'agit en réalité d'un pointeur.

L'action d'utiliser des [] même vide mais avec des valeurs comme le dernier exemple envoie le message suivant à l'ordinateur ;

« Reserve dans ta mémoire de l'espace pour X element du type <insert_your_type> »

L'ordinateur va nous donner en retour de l'espace mémoire, mais il nous présente l'information comme les variables en adresse dans sa logique.

```
Parrot Terminal
File Edit View Search Terminal Help
1 #include <stdio.h>$
2 $
3 int main(){$
4 int tableau[]={10,20,30};$
5 $
6 printf("l'adresse du tableau est %p\n",tableau) $
7 printf("l'adresse du premier element du tableau %p\n",&tableau[0]) $
8 }$

Parrot Terminal
File Edit View Search Terminal Help
~ [user@parrot]~-[~/Desktop]
~ $ ./array
~ l'adresse du tableau est 0x7ffd810223d4 IDENTIQUE
~ l'adresse du premier element du tableau 0x7ffd810223d4
~ [user@parrot]~-[~/Desktop]
~ $
```

On voit que le premier élément de la liste est au même endroit que le tableau en soit.
A noter que dans un cas j'ai juste mis le nom du tableau (qui est tableau dans l'image) et dans l'autre j'ai utilisé le &. On a déjà vu le & dans l'explication des pointeurs.

Ca veut dire aussi que l'on peut aussi trouver l'élément du tableau en utilisant le concept de dereferencement vu avec les pointeurs. C'est-à-dire utiliser * avant le pointeur pour accéder aux valeurs. Ici on le met entre parenthèse avec un +1 car on souhaite accéder au second éléments du tableau. Cela peut être pratique lorsque l'on veut faire des boucles dans une array.

```
1 #include <stdio.h>$
2 $
3 int main(){
4     int tableau[]={10,20,30};
5 }
6 printf("la seconde valeur du tableau est %d\n",tableau[1]);
7 printf("la seconde valeur du tableau est %d\n",*(tableau+1));
8 }
```

Parrot Terminal

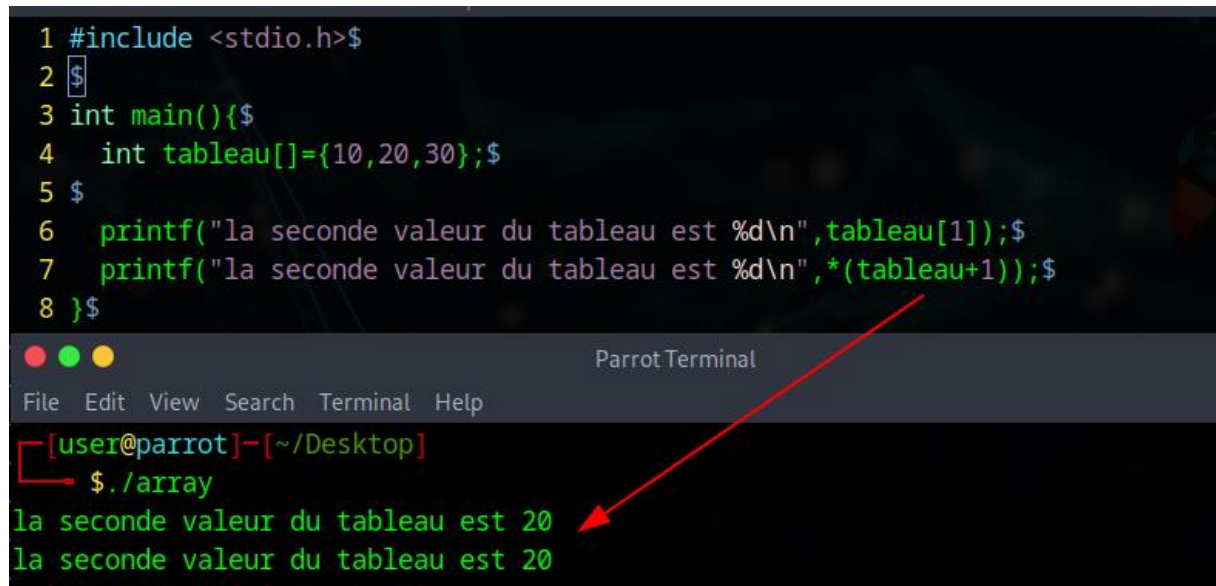
File Edit View Search Terminal Help

[user@parrot]-[~/Desktop]

→ ./array

la seconde valeur du tableau est 20

la seconde valeur du tableau est 20



CHAR

Attention, il ne peut enregistrer qu'une lettre !

Attention, lors de la déclaration de la variable il faut utiliser ' et pas « »

Si vous essayez de sortir avec le printf() %d => vous aurez la valeur ASCII de la lettre. N'oubliez pas l'ordinateur n'enregistre pas vraiment la lettre mais la valeur ASCII de celle-ci.

Pour avoir la lettre « lisible » / « convertit » => il faut utiliser %c

```
1 #include <stdio.h>$
2 $
3 $ gcc program.c -o program
4 int main(){
5     char lettre = 'A';
6     printf("la valeur ASCII de lettre est %d\n",lettre);
7     printf("la lettre est %c\n",lettre);
8 }
9 $ ./program
10 $ lettre 65
```

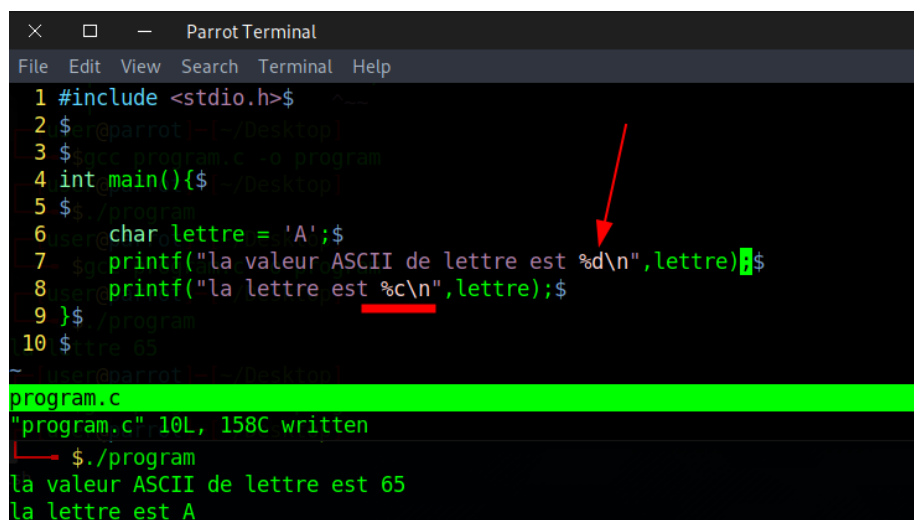
program.c

"program.c": 10L, 158C written

→ ./program

la valeur ASCII de lettre est 65

la lettre est A



<https://www.ascii-code.com/fr>

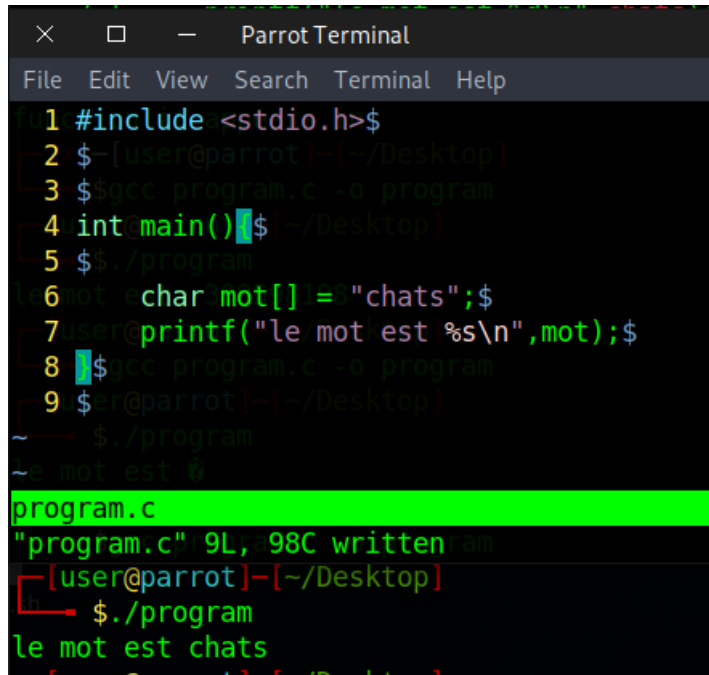
65	101	41	01000001	A	A	A majuscule
----	-----	----	----------	---	-------	-------------

Chaine de caractères

Comment faire pour enregistrer un mot ou un texte, si on ne peut pas faire qu'une lettre ?

Grace au tableau ! Une chaine de caractère = un tableau

C'est ici qu'on peut utiliser « » et non pas " pour enregistrer un mot/phrase/texte



```
Parrot Terminal
File Edit View Search Terminal Help
1 #include <stdio.h>
2 $-[user@parrot]~[~/Desktop]
3 $gcc program.c -o program
4 int main() {
5     char mot[] = "chats";
6     printf("le mot est %s\n", mot);
7     $gcc program.c -o program
8     $[user@parrot]~[~/Desktop]
9     $./program
~ le mot est chats
program.c
"program.c" 9L, 98C written on disk
[user@parrot]~[~/Desktop]
$ ./program
le mot est chats
```

A noter que pour printf() notre chaine de caractère => on utilise %s

ATTENTION il y a un piège avec les chaines de caractères !!!

L'ordinateur enregistre une valeur en plus ! il enregistre \0 en dernier élément pour indiquer que la chaine de caractère s'arrête ici.

Dans la méthode que je vous ai montré, vous n'avez pas à le mettre. Mais si vous enregistré une chaine de caractère lettre par lettre, il faut penser à la dernier \0

```
File Edit View Search Terminal Help
5 $er@parrot|~|~/Desktop|
6- $. ./char mot[5]; //ici je met 5$
7 mot mot[0] = 'c'; $
8 user mot[1] = 'h'; $ktop|
9- $g mot[2] = 'a'; $ program
10 user mot[3] = 't'; $ktop|
11- $. ./mot[4] = '\0'; //je signale que c'est la fin$
12 $t est chat
13 dern printf("le mot est %s\n", mot); $
14 user printf("le dernier char est %c\n", mot[4]); $
15 } $ gcc program.c -o program
program.c
$ ./program
le mot est chat
le dernier char est
```

En soit cela n'est pas obligatoire mais recommandé. Pourquoi ? Pas obligatoire car j'ai bien dit mot[6] donc il y a l'espace qui suffit. Par contre si je mets mot[5] et pas de \0

```
Parrot Terminal
File Edit View Search Terminal Help
5 $er@parrot|~|~/Desktop|
6- $. char mot[4]; //ici je met 4$
7 mot mot[0] = 'c'; $
8 dern mot[1] = 'h'; $
9 user mot[2] = 'a'; $ktop|
10- $g mot[3] = 't'; $ program
11 user //je ne signale pas que c'est la fin$
12 $ $. ./program
13 mot printf("le mot est %s\n", mot); $
14 } $ nier char est
15 $er@parrot|~|~/Desktop|
program.c
"program.c" 15L, 217C written
$ ./program
le mot est chatpe 
V
```

Ceci est aussi « non obligatoire » car cela dépend du contexte de l'utilisation. Techniquement ici je mélange un peu une chaîne de caractère et un string. Il y a une distinction du a la presence du \0. Mais disons que dans ce document j'ignore volontairement cela dans un soucis de simplification, **take it or leave it**.

Pour plus d'opération sur des chaînes de caractères, il faut utiliser la bibliothèque string.h

```
#import <string.h>
```

Listes non exhaustive de fonction ;

strlen pour calculer la longueur d'une chaîne.

strcpy pour copier une chaîne dans une autre.

strcat pour concaténer 2 chaînes.

strcmp pour comparer 2 chaînes.

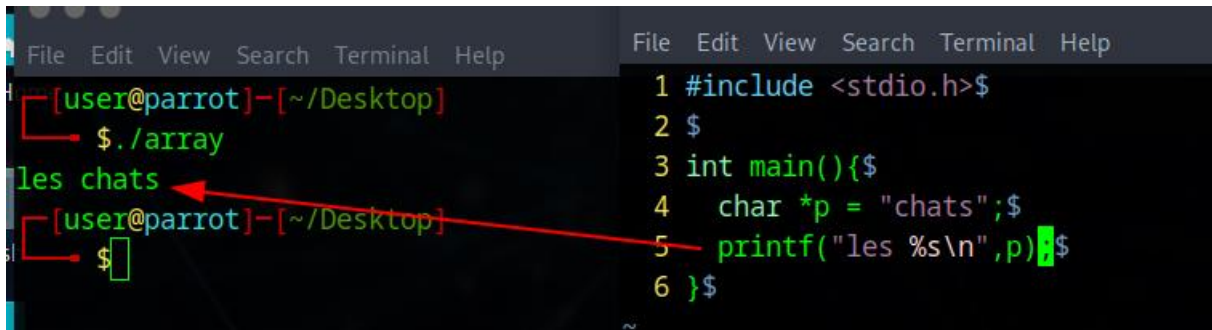
strchr pour rechercher un caractère.

strpbrk pour rechercher le premier caractère de la liste.

strstr pour rechercher une chaîne dans une autre.

sprintf pour écrire dans une chaîne.

Autre méthode de déclaration :



```
File Edit View Search Terminal Help
1 #include <stdio.h>$
2 $
3 int main(){
4   char *p = "chats";
5   printf("les %s\n",p);
6 }$

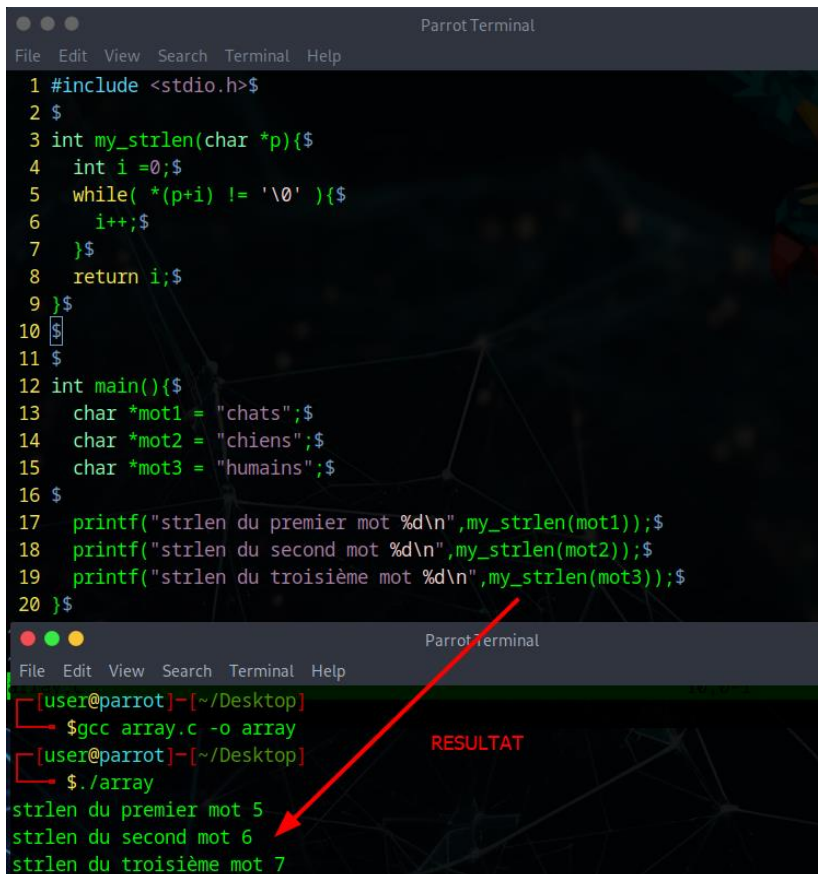
[user@parrot]~/Desktop
$ ./array
les chats
[user@parrot]~/Desktop
$
```

N'oublions pas que dès qu'on parle d'une chaîne de caractère, on parle d'un tableau / array. Cela veut dire aussi qu'on peut déclarer un string à la manière d'un pointeur. Vous pourrez quand même accéder à chaque élément avec les []

Exemple ; création d'une fonction strlen like

Strlen est une fonction qui compte la longueur d'un string.

On reproduit cette fonction ;



```
Parrot Terminal
File Edit View Search Terminal Help
1 #include <stdio.h>$
2 $
3 int my_strlen(char *p){
4   int i = 0;
5   while( *(p+i) != '\0' ){
6     i++;
7   }
8   return i;
9 }$
10 $
11 $
12 int main(){
13   char *mot1 = "chats";
14   char *mot2 = "chiens";
15   char *mot3 = "humains";
16   $
17   printf("strlen du premier mot %d\n",my_strlen(mot1));
18   printf("strlen du second mot %d\n",my_strlen(mot2));
19   printf("strlen du troisième mot %d\n",my_strlen(mot3));
20 }$

[user@parrot]~/Desktop
$ gcc array.c -o array
[user@parrot]~/Desktop
$ ./array
strlen du premier mot 5
strlen du second mot 6
strlen du troisième mot 7
```

Commentaire sur ce code ;

-Pas de prototype ici. Directement déclaration de la fonction avant main() pour que cela fonctionne.

-On a souvent vu des cas avec return 0 ; pour des fonctions de type int. Ici ce n'est pas le cas car je veux retourner la longueur du mot. En faisant de cette manière cela pourrait être utiliser en créant une variable int puis en mettant le resultat de cette fonction directement dans la variable. Ici par simplicité, j'ai tout de suite mis le resultat dans le printf().

-Dans la fonction my_strlen() => j'ai mis une boucle qui et dans la condition il y a *(p+i)
On utilise ici l'idée de dereferencement dont on a parlé avec les pointeur et les arrays. P étant l'adresse de la chaine string.

P+i quand i vaut 0 => premier caractère.

P+i quand i vaut 1 => second caractère.

P+i quand i vaut.....

-Toujours dans la mise en place de la boucle ; on compare donc chaque element du la chaine de caractère à '\0'. Il s'agit pour rappel du null operator qui précise la fin de la chaine.

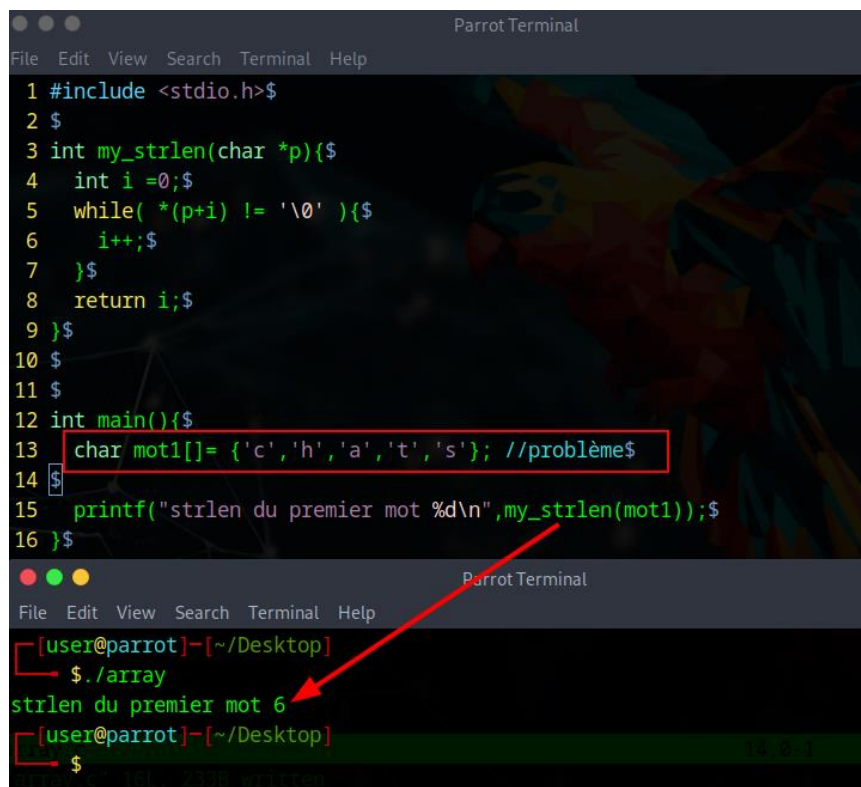
-Pas besoin de mettre une autre variable pour compter, on gagne une ligne comme ca. Puisque i va atteindre la bonne valeur quoi qu'il arrive.

-Dans le printf() vous pouvez constater que je n'ai pas mis de & comme on a pu le voir dans les pointeurs. Pourquoi ?

En indiquant

-pour rappel ++ ; permet d'incrémenter i pour ne pas être dans une boucle infini + compter + changer d'element du string.

Ici vous pouvez aussi vous rendre compte que ma simplification « **take it or leave it** » précédente vient nous poser problème. Dans le cas ou on a pas de \0, on fait quoi ?



```
1 #include <stdio.h>$
2 $
3 int my_strlen(char *p){$
4     int i =0;$
5     while( *(p+i) != '\0' ){$
6         i++;$
7     }$
8     return i;$
9 }$
10 $
11 $
12 int main(){$
13     char mot1[] = {'c','h','a','t','s'}; //problème$
14     $
15     printf("strlen du premier mot %d\n",my_strlen(mot1));$
16 }$

[user@parrot]-[~/Desktop]
$ ./array
strlen du premier mot 6
[user@parrot]-[~/Desktop]
$
```

Ici on a clairement pas le résultat qu'on souhaite !

Du coup on peut même se poser la question. Mais comment je fais pour un strlen mais pas d'un str (string) mais d'une longueur d'un tableau ?

Chaque problème à sa solution ;

Sizeof()

Sizeof() est un opérateur qui permet de connaître la taille en octet d'une variable ou d'un type de variable.

Celui-ci est exprimé en octet (bytes est le mot équivalent en anglais), c'est-à-dire 8 bits.

```
int main()$
{$
  es printf("la taille en octet d'une variable int est de %d octet\n",sizeof(int));$
  es printf("la taille en octet d'une variable long est de %d octet\n",sizeof(long));$
  es printf("la taille en octet d'une variable double est de %d octet\n",sizeof(double));$
  es printf("la taille en octet d'une variable char est de %d octet\n",sizeof(char));$
  $
}$
```

```
[user@parrot]~[~/Desktop]
$ ./program
la taille en octet d'une variable int est de 4 octet
la taille en octet d'une variable long est de 8 octet
la taille en octet d'une variable double est de 8 octet
la taille en octet d'une variable char est de 1 octet
```

ATTENTION ; Techniquement sizeof renvoie du size_t et pas un int. Dans certains cas ça pourrait poser problème. Voulez le mettre quand même dans un int, mais méfiance parfois.

<https://noncombatant.org/2023/02/12/int-size-t/#:~:text=So%20if%20the%20subscript%20type,counts%2C%20and%20indices%2Fsubscripts>

L'utilisation principale de sizeof() est dans l'allocation de mémoire dynamique. Mais avant ça, on va l'utiliser pour répondre à notre précédent problème ;

Comment connaître la longueur d'un array ?

```
1 #include <stdio.h>$
2 $
3 int main(){$
4   //declaration array$
5   int int_array[] = {10,20,30};$
6   float float_array[] = {5.55,6.66,7.77,8.88,9.99,10.00,11.11};$
7 $
8   //taille de l'array + taille d'un element => nous donne le nb d'element$
9   printf("la taille de int_array est de %zu octet\n",sizeof(int_array));$
10  printf("la taille d'un element de int_array %zu octet\n",sizeof(int_array[0]));$
11 ++$
12  size_t long_int_array = sizeof(int_array)/sizeof(int_array[0]);$
13  printf("il y a %zu elements dans int_array\n",long_int_array);$
14 $
15  //taille de l'array + taille d'un element => nous donne le nb$
16  printf("la taille de float_array est de %zu octet\n",sizeof(float_array));$
17  printf("la taille d'un element de float_array est de %zu octet\n",sizeof(float_array[0]));$
18 $
19  size_t long_float_array = sizeof(float_array)/sizeof(float_array[0]);$
20  printf("il y a %zu elements dans float_array\n",long_float_array);$
21 $
```

Resultat ;

```
13- $ ./array "il y a %zu elements dans int_array\n", long_int_array
la taille de int_array est de 12 octet
la taille d'un element de int_array 4 octetement => nous donne le
il y a 3 elements dans int_array_array est de %zu octet\n", sizeof
la taille de float_array est de 28 octetfloat_array est de %zu oc
la taille d'un element de float_array est de 4 octet
il y a 7 elements dans float_arraysizeof(float_array)/sizeof(float
```

On peut donc déterminer la longueur d'un array en prenant la taille et la diviser par la taille d'un seul element. Comme un array est composé toujours du même type on retrouve quoi qu'il arrive le bon resultat.

De plus on trouve aussi le bon résultat sur n'importe quel ordinateur et OS ! Ce qui est important en fonction de si vous êtes sur du 32 bit ou 64 bit. C'est d'ailleurs tout la problématique de l'allocation de mémoire dynamique.

Allocation dynamique / Malloc()

Pourquoi faire de l'allocation dynamique de mémoire ?

Quand on crée une variable, l'ordinateur nous donne un endroit dans la mémoire pour enregistrer ensuite une valeur, un char.....

```
1 #include <stdio.h>$
2 //la taille en octet d'une variable long est de 8 octet
3 int main()$
4 {$
5     int variable; //ici l'ordinateur libère un espace pour cette variable$
6     //on peut d'ailleurs déjà aller chercher son adresse$
7     printf("%p\n",&variable);$
8 $
9 //on peut aussi savoir combien il prend d'espace$
10 printf("la variable prend %d bytes\n",sizeof(variable));$
11 $
12 }$
$ ./program
0x7fff5314961c
la variable prend 4 bytes
```

Rappel ; bytes = octets, 1 (un) (uno) (eins) (yí) (๑) octet = 8 bits

Cette mémoire n'est pas illimitée, même si on en a normalement beaucoup sur notre ordinateur.

Comment utiliser malloc ?

```
1 #include <stdio.h>$
2 #include <stdlib.h> //c'est ici qu'est malloc$
3 $
4 int main(){$
5     int nb; // nb de case$
6     printf("combien de case dans mon tableau ?\n");$
7     scanf("%d",&nb);$
8 $
9     int *tableau; // pointeur qui va me permettre de faire mon array plus tard, il va contenir des int$
10    tableau = malloc(nb*sizeof(int)); // je libère de l'espace pour mon tableau$
11    $
12    free(tableau); // on doit free l'allocation de mémoire done$
13 $
14    return 0;$
15 }$
16 $
```

STRUCTURE, ENUMERATION & UNION

Une structure est un assemblage de variables qui peuvent avoir différents types. Pour l'utiliser on le déclare dans un fichier header .h (voir plus haut sur les projets).

Comment déclarer une structure ?

```
struct <nom_structure>{
    int <variable>;
    float <variable>;
    char <variable>;
};
```

A noter, vous pouvez mettre des tableaux/array dans les structures. On a aussi terminer notre structure par un ; à ne pas oublier (ce n'est pas en lien avec le fait que j'ai fait cette erreur...je vois pas de quoi vous parler)

Techniquement on est pas dans l'obligation de mettre la structure dans le .h. Mais on le fait très souvent par simplicité dans les différents projets.

Exemple ;

Fichier en .h

```
1 #include <stdio.h>$
2 struct test_structure{$
3     int hauteur;$
4     int longueur;$
5     char lettre;$
6 };$
```

Notre program.c qui est le main()

```

1 #include <stdio.h>$
2 #include "structure.h"$
3 $
4 int main(){
5     struct test_structure variable_structure; //ici on crée une variable "variable_structure" qui n'est donc pas int/float/long/double....mais un struct$
6     variable_structure.hauteur = 10; //ici on va mettre des valeurs dans notre structure, a noter la forme <structure>.<element>+$
7     variable_structure.longueur = 1;$
8     variable_structure.lettre = 'A';$
9     printf("la hauteur est de %d, la longueur est de %d et la lettre est %c\n", variable_structure.hauteur, variable_structure.longueur, variable_structure.lettre);$
10 $
11 $
12 $
13 $

```

Resultat

```

[~]-[user@parrot]-[~/Desktop]
$ gcc program.c structure.h -o program
[user@parrot]-[~/Desktop]
$ ./program
la hauteur est de 10, la longueur est de 1 et la lettre est A
[user@parrot]-[~/Desktop]
$

```

Pour passer des valeurs dans la structure, on peut aussi ainsi ;

```

1 #include <stdio.h>$
2 #include "structure.h"$
3 $
4 int main(){
5     struct test_structure variable_structure = {10,1,'B'}; //ici on passe les valeurs de la création$
6 $
7     printf("la hauteur est de %d, la longueur est de %d et la lettre %c\n", variable_structure.hauteur, variable_structure.longueur, variable_structure.lettre);$
8 $
9 $
10 $

```

Résultat ;

```

[user@parrot]-[~/Desktop]
$ gcc program.c structure.h -o program
[user@parrot]-[~/Desktop]
$ ./program
la hauteur est de 10, la longueur est de 1 et la lettre B
[user@parrot]-[~/Desktop]
$

```

La structure présente aussi un avantage ; vous pouvez depuis une fonction retourner plusieurs résultats en les mettant dans une structure. A la fin de votre fonction vous pouvez faire un return <structure_que_vous_avez_créé> ;

Autre exemple d'initialisation d'une structure ;

```
File Edit View Search Terminal Help
1 #include <stdio.h>$
2 $
3 int main(){
4     //creation structure$
5     struct struct_init{$
6         int x;$
7         int z;$
8         float y;$
9     };$
10 $
11     struct struct_init new_struct= {$
12         .x = 1,$
13         .z = 2,$
14         .y = 3.1$
15     };$
16 $
17     printf("%d\n",new_struct.x);$
18     printf("%f\n",new_struct.y);$
19 }$
```

[user@parrot]-[~/Desktop]
\$./struct_init

1
3.100000

Ici on utilise .<element_de_la_structure> suivi d'une virgule.

Structure and array :

On peut bien sur utiliser un array dans une structure.

```
[user@parrot]-[~/Desktop/project]
$ gcc test.c -o test
[user@parrot]-[~/Desktop/project]
$ ./test
3
2
[user@parrot]-[~/Desktop/project]
$
```

```
1 #include <stdio.h>$
2 $
3 struct point{$
4     int a;$
5     int b;$
6 };$
7 $
8 int main(){
9     struct point test[5]; //un array dans une structure$
10    test[0].a = 1;$
11    test[0].b = 2;$
12    test[1].a = 3;$
13    test[1].b = 4;$
14    printf("%d\n",test[1].a);$
15    printf("%d\n",test[0].b);$
16    return 0;$
17 }$
```

Typedef

Typedef permet d'éviter de faire une aussi longue declaration dans main().

Il faut l'indiquer dans votre fichier .header.

```
typedef struct <nom_de_la_structure> <alias_de_la_structure> ;
```

```

1 #include <stdio.h>$
2 typedef struct test_structure structuro;$
3 $
4 struct test_structure{$
5     int hauteur;$
6     int longueur;$
7     char lettre;$
8 };$

```

Il n'y avait pas besoin de mettre <stdio.h> dans le structure.h

Dans main()

```

1 #include <stdio.h>$
2 #include "structure.h"$
3 $
4 int main(){
5     structuro variable_structure = {10,1,'B'}; //ici c'est structuro maintenant, plus besoin de mettre struct$
6     printf("la hauteur est de %d, la longueur est de %d et la lettre %c\n", variable_structure.hauteur, variable_stru
7 }

```

Enumerations

L'énumération ressemble à la structure dans l'organisation => c'est-à-dire a mettre dans un fichier .h et la définition (même l'utilisation de typedef).

Mais l'énumération est juste une liste de valeur possible.

Comment le déclarer ?

```

enum <nom_enumeration> {
    <valeur1>, <valeur2>, <valeur3>....<valeur n>
}

```

Noter la différence avec les structures sur la déclarations avec les virgules.

```

File Edit View Search Terminal Help
[~]-[user@parrot]-[~/Desktop]
$ gcc enumeration.c -o enumeration
[user@parrot]-[~/Desktop]
$ ./enumeration
la valeur de variable0 = 0
la valeur de variable1 = 4
[user@parrot]-[~/Desktop]
$

1 #include <stdio.h>
2
3 typedef enum jours {
4     lundi,
5     mardi,
6     mercredi,
7     jeudi,
8     vendredi,
9     samedi,
10    dimanche,
11 } jours_t;
12
13 int main(){
14     //variable qui va prendre la val
15     jours_t variable0 = lundi;
16     jours_t variable1 = vendredi;
17
18     printf("la valeur de variable0 = %d\n",variable0);
19     printf("la valeur de variable1 = %d\n",variable1);
20 }

```

Je ne récupère pas le « mot » conservé mais la position du « mot ». Cela peut être utile pour gérer des erreurs à des endroits dans votre code.

Vous pouvez aussi mettre des valeurs spécifiques ;

```
home > user > Desktop > C enumeration.c
1 #include <stdio.h>
2
3 typedef enum jours {
4     lundi = 1337,
5     mardi,
6     mercredi,
7     jeudi,
8     vendredi = 42,
9     samedi,
10    dimanche,
11 } jours_t;
12
13 int main(){
14     //variable qui va prendre la val
15     jours_t variable0 = lundi;
16     jours_t variable1 = vendredi;
17
18     printf("la valeur de variable0 = %d\n",variable0);
19     printf("la valeur de variable1 = %d\n",variable1);
20 }
```

```
[user@parrot] ~/Desktop
$ gcc enumeration.c -o enumeration
$ ./enumeration
la valeur de variable0 = 0
la valeur de variable1 = 4
[user@parrot] ~/Desktop
$ gcc enumeration.c -o enumeration
$ ./enumeration
la valeur de variable0 = 1337
la valeur de variable1 = 42
[user@parrot] ~/Desktop
$
```

UNIONS

L'union c'est l'union des structures et des énumérations.

```
union <nom_union> {
    <type_variable> <nom_variable>;
    <type_variable> <nom_variable>;
    .....
    .....
};
```

Cela ressemble beaucoup à une structure car on peut avoir plusieurs types de variable.
L'assignation de valeur fonctionne de la même manière qu'une structure;

```
[user@parrot] ~/Desktop
$ gcc program.c -o program
$ ./program
bernardo
[user@parrot] ~/Desktop
$
```

```
1 #include <stdio.h>$
2 $
3 union chats{$
4     char *name;$
5     int age;$
6 };$
7 $
8 int main(){$
9     union chats chaton = {$
10         chaton.name = "bernardo"$
11     };$
12 $
13 printf("%s\n",chaton.name);$
14 }$
```

Mais c'est ici où les différences commencent aussi ! Vous ne pouvez que mettre une valeur.
Vous ne pouvez pas donner un autre attribut, sinon il s'agit d'une structure et pas d'une union.

```
[user@parrot] ~/Desktop
$ gcc program.c -o program
$ ./program
bernardo
[user@parrot] ~/Desktop
$ gcc program.c -o program
program.c: In function 'main':
program.c:11:9: warning: excess elements in union initializer
11 |     chaton.age = 4
    |     ^~~~~~
program.c:11:9: note: (near initialization for 'chaton')
[user@parrot] ~/Desktop
$
```

```
1 #include <stdio.h>
2 $
3 union chats{$
4     char *name;$
5     int age;$
6 };$
7 $
8 int main(){
9     union chats chaton = {$
10         chaton.name = "bernardo",
11         chaton.age = 4$
12     };$
13 $
14 printf("%s\n",chaton.name);$
15 }
```

On peut bien sûr utiliser typedef sur les enums ;

```

1 #include <stdio.h>$
2 $
3 typedef union chats{$
4     char *name;$
5     int age;$
6 } chats_u;$
7 $
8 int main(){$
9     chats_u chaton = {$
10         chaton.name = "bernardo"$
11     };$
12 $
13     printf("%s\n", chaton.name);$
14 }$

```

Après vous pouvez mettre des Unions dans des structs (et inversement) avec des pointeurs dans des fonctions.....ca sert surement a rien.....mais vous pouvez le faire et dire à tout le monde « je l'ai fait, je suis meilleur que vous »

Ici aussi il faut faire attention à la mémoire utilisé.

-La taille d'un union = la taille du plus gros éléments possible

-Si vous essayez d'accéder à un des éléments non utilisé ca fout le bordel, et quelqu'un va vous tirer dessus ! (Crazy chiitan japonese mascot)

Préprocesseur

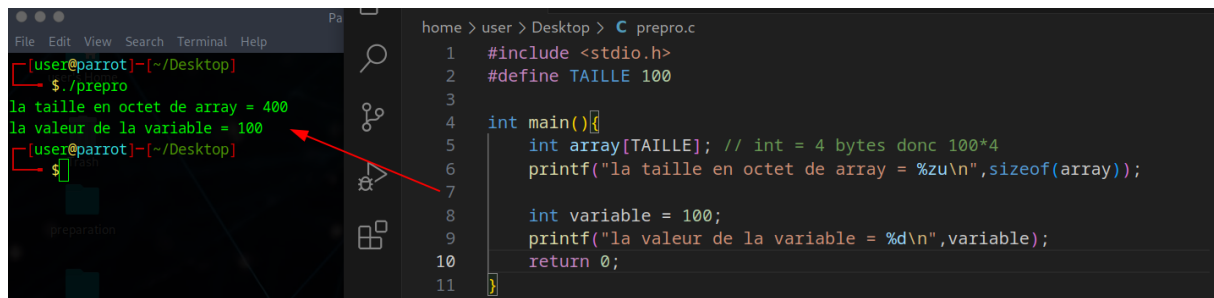
Directive du préprocesseur = lignes qui commencent avec #

On connaît déjà avec le #include qui nous permet d'ajouter des librairies ou nos propres headers file.

Il s'agit en réalité d'une étape lors de la compilation de votre programme. Qui « préparer » tout ce qui commence par un #.

#define permet d'associer un mot à une valeur. Cela va s'appliquer à tout votre fichier, sans prendre de place en mémoire à l'inverse d'une constante.

On l'utilise souvent pour des tableaux / array



```
home > user > Desktop > C prepro.c
1  #include <stdio.h>
2  #define TAILLE 100
3
4  int main()
5  {
6      int array[TAILLE]; // int = 4 bytes donc 100*4
7      printf("la taille en octet de array = %zu\n",sizeof(array));
8
9      int variable = 100;
10     printf("la valeur de la variable = %d\n",variable);
11     return 0;
12 }
```

```
[user@parrot]~[/Desktop]
$ ./prepro
la taille en octet de array = 400
la valeur de la variable = 100
[user@parrot]~[/Desktop]
$
```

MAIS ATTENTION => c'est ici qu'il peut y avoir énormément d'erreur de dépassement de la mémoire.

Vous pouvez aussi faire des calcul dans **#define** à partir d'autres valeurs que vous avez aussi **#define**.

Dans le préprocesseur, il existe aussi des constantes déjà existantes ;

__LINE__ donne le numéro de la ligne actuelle.

__FILE__ donne le nom du fichier actuel.

__DATE__ donne la date de la compilation.

__TIME__ donne l'heure de la compilation.

Macros

Vous pouvez définir des macros => c'est-à-dire remplacer une fonction avec des paramètres par un autre

#define <fonction()> <fonction_qui_existe_avec_paramètre> ;

MANIPULATION DE FICHIER

En plus de stdio.h, on doit utiliser stdlib.h

Avant d'ouvrir un fichier il faut déclarer un pointeur sur FILE. Pourquoi ? Parce que c'est comme ça qu'on le retrouve grâce à stdio.h.

```
FILE *fichier = NULL ;
```

Ensuite on va pouvoir utiliser fopen() qui est la fonction qui permet d'ouvrir notre fichier ;

```
fichier = fopen(« <nom_du_fichier> », « mode » );
```

Mode ici permet de déterminer ce qu'on va faire avec ce fichier ;

r pour read only => le fichier doit déjà exister

w pour écriture seulement => si le fichier n'existe pas il est créé

a pour ajout => ajoute à la fin du document les informations

a+ pour ajout en lecture / écriture à la fin => écriture + lecture à la fin

r+ pour lecture et écriture => le fichier doit déjà exister

w+ pour lecture et écriture + suppression du contenu au préalable

Nom_du_fichier c'est s'il est dans le même dossier/directory, sinon il faut mettre le chemin d'accès (attention il faut mettre 2 \ sinon le compilateur va penser à \n).

Ecrire dans un fichier

On peut le faire à travers différentes fonctions ;

fputc() pour mettre un seul caractère

fputs() pour mettre une chaîne de caractère

fprintf() pour mettre une chaîne formatée comme printf() qu'on connaît

Fermer un fichier

Il faut fermer le fichier une fois qu'on a fait nos modifications avec *fclose(<fichier>)* ;

```
1 #include <stdio.h>$
2 #include <stdlib.h>$
3 $
4 int main(){
5     FILE *fichier = NULL; // le pointeur
6     fichier = fopen("test.txt","r+");//on ouvre le fichier test.txt
7     printf("How many cats?\n");
8     int cats;
9     scanf("%d",&cats);
10    fprintf(fichier,"le gars il a %d chats",cats);//on print directement dans le fichier
11    fclose(fichier);
12    return 0;
13 }
```

```
[user@parrot]-[~/Desktop]
└─$ cat test.txt
[user@parrot]-[~/Desktop]
└─$ gcc program.c -o program
[user@parrot]-[~/Desktop]
└─$ ./program
How many cats?
5
[user@parrot]-[~/Desktop]
└─$ cat test.txt
le gars il a 5 chats
[user@parrot]-[~/Desktop]
└─$
```

UTILISATION DE GDB

Vidéo youtube que j'ai réalisé sur GDB = <https://www.youtube.com/watch?v=Sq3tFv7Ux5w>

GDB = GNU Debugger comme GCC est le GNU compiler collection.

C'est un outil qui va nous permettre de debugger nos programmes en lançant le programme.

Quand vous codez, en fonction de l'IDE que vous utilisez il y a parfois déjà des debuggers présents à l'intérieur.

L'avantage c'est que GDB est présent sur Linux par défaut dans la plupart des cas.

De plus GDB est aussi souvent utilisé dans un autre domaine proche ; le reverse engineering. Dans les différents CTF (= *capture the flag* challenge de cybersécurité où il faut trouver un flag pour prouver la résolution du challenge) on peut être amené à utiliser GDB pour déconstruire un fichier compilé en C et analyser l'assembleur (= *assembly* instruction au CPU).

Pour utiliser GDB en tant que debugger il faut lors de la compilation ajouter un paramètre ; -g
Exemple ; **gcc program.c -o program -g**

Avec GDB qu'est-ce qu'on peut faire ? On peut executer notre programme et le suivre ligne par ligne mais aussi ;

- regarder les fonctions
- voir les valeurs des variables pendant l'exécution du programme
- voir les registres à l'intérieur du CPU
- mettre en place des breakpoints => lancer le programme jusqu'au point (le breakpoint) qu'on a mis en place. Il s'arrête alors au moment de l'exécution>>
- On peut même examiner la mémoire et registre du CPU.

Avec GDB on va être capable de suivre un programme ligne par ligne pendant son exécution, de regarder à l'intérieur des fonctions, de voir les valeurs des variables et les changements qui ont eu lieu, mettre

REMARQUE IMPORTANTE ; toutes les commandes que l'on va voir vont être écrites « en complet » pour quasiment toutes les commandes il existe un raccourci. Par exemple ; next => n

Par soucis de clarté on va écrire le mot complet à chaque fois et pas une lettre.

Lancement de GDB ;

gdb ./<nom_de_votre_programme>

<nom_de_votre_programme> On parle ici de ce qui est compilé, pas du code source

Après lancement, début de l'utilisation ;

Une fois GDB lancé, vous pouvez taper la commande **list** pour voir où vous en êtes dans votre programme.

Ensuite pour lancer votre programme, vous pouvez utiliser la commande **start**

Pour passer à la prochaine ligne, vous avez 2 possibilités mais qui n'agissent pas de la même manière ; **step** et **next**

step continue l'exécution jusqu'à la nouvelle ligne et s'arrête, il step in. On peut voir ce qui va se passer après l'exécution de la ligne.

next continue l'exécution mais on a exécuté la ligne en question.

C'est une notion importante quand on va arriver sur des fonctions. Step vous fait entrer DANS la fonction alors que next va l'exécuter. Step est donc intéressant pour une fonction qu'on a fait....mais pas pour des printf(), write() etc....

Si vous vous trouvez dans une fonction mais que vous n'avez pas besoin d'aller au bout ; **finish**

info locals sort les variables locales.

info args va vous montrer les arguments passés à une fonction

Si vous voulez voir la valeur d'une variable **print <nom_de_votre_variable>**

Si vous voulez surveiller une variable pendant tout le long de l'exécution **watch** **<nom_de_votre_variable>**

Breakpoint, va mettre en pause notre programme dans l'endroit spécifié ;
break <la_ou_on_veut_s_arreter>

Le lieu ou on va vouloir s'arrêter, cela peut être un numéro de ligne ou une fonction. On peut donc mettre un breakpoint à main() par exemple.

Idéalement on place le breakpoint avant l'endroit ou le bug se produit.

Une fois le breakpoint mis en place, on va utiliser **run** pour arriver jusqu'à notre point (et pas start).

Pour quitter GDB on utilise la commande **quit**

Si vous étiez entrain de l'utiliser en lançant un programme, vous devez sûrement mettre yes GDB ne fait pas de sauvegarde, vous ne pourrez pas reprendre exactement la ou vous en étiez sauf si vous mettez un breakpoint lors du lancement du prochain lancement de GDB.

Vous trouvez une cheat sheet ici ; <https://darkdust.net/files/GDB%20Cheat%20Sheet.pdf>
Un autre site avec plusieurs commandes ; <https://visualgdb.com/gdbreference/commands/>

Partie moins intéressante pour certains, ici c'est pour de l'utilisation dans du reverse ;

Dans ce genre de challenge, le code n'utilise pas forcément l'option -g de gcc

layout next va vous permettre de voir les instructions en assembleur (ou vous faire passer pour un monstre en code)

break main est souvent utilisé en reverse pour atterrir à l'endroit qu'il faut sans se taper plein de choses inintéressantes

info registers va vous montrer les valeurs dans les registres (CPU).

Si vous recherchez un registre en particulier, il faut mettre le signe \$ devant pour l'utiliser
Exemple ; \$eax

L'utilisation de run, step, break etc...reste similaire.

Il arrive des cas ou voir le contenu des registres est intéressant pour debugger votre programme. Un petit exemple ; je fais un exercice sur codewars, dans celui-ci je dois bouclé sur un string pour modifier des valeurs. Il se trouve que mon algorithme est pas bien fait pour diverses raisons mais le programme compile + fonctionne mais a un resultat étrange car il ne return qu'une chaine string incomplète.

C'est en regardant les registres que je me suis rendu compte du problème. Je fais -32 dans certains cas => et cela arrive quand je traite l'espace....sauf que espace en ASCII = 32, donc 32-32 = 0....donc mon algorithme remplace l'espace par NULL '\0'. Et en regardant le registre 'eax' au fur et a mesure de l'exécution de mon programme j'ai compris que j'étais une MERDE.

ASTUCES

Convertir un int en float/double

On peut cast c'est-à-dire changer le type de variable d'un int vers double/float

```
1 #include <stdio.h>$
2 $
3 int main(){$
4     int a= 1;$
5     double b;$
6     printf("%d\n",a); // int$
7     b = (double) a; // cast => transformation de notre int en double$
8     printf("%lf\n",b);$
9 }
```

```
[user@parrot]--[~/Desktop]
└─$ ./program
1
1.000000
```

On peut faire ça dans l'autre sens float => vers int de la même manière.

On peut aussi le faire dans un calcul (mais attention au arrondi dans ce cas là)

resultat = (int) money/price ;

Ici resultat est int, money et price sont double.

Enregistrer plusieurs valeurs par un scanf()

On peut tout de suite mettre plusieurs valeurs dans un scanf()

```
1 #include <stdio.h>$
2 $
3 int main(){$
4     int a, b, c;$
5     printf("entre 3 valeurs\n");$
6     scanf("%d %d %d",&a,&b,&c);$
7     printf("valeur a %d\nvaleur b %d\nvaleur c %d\n",a,b,c);$
8 $
9 }
```

```

[user@parrot]--[~/Desktop]
└─$ ./program
entre 3 valeurs
45
58
12
valeur a 45
valeur b 58
valeur c 12
[user@parrot]--[~/Desktop]
└─$ ./program
entre 3 valeurs
45 12 63
valeur a 45
valeur b 12
valeur c 63

```

A noter on peut mettre les valeurs soit en appuyant sur *entrée* soit en les séparant par un espace

[Enregistrer depuis un scanf\(\) une valeur dans un tableau / array](#)

```

1 #include <stdio.h>
2 $
3 int main(){
4     int ingredient[10], id, i;
5     for(i=0;i<10;i++){
6         scanf("%d",&ingredient[i]);
7     }
8     scanf("%d",&id);
9     printf("%d",ingredient[id]);
10 }

```

Pas besoin de passer par une variable en plus qui ferait dans notre exemple ;

```
scanf(« %d », &variable_temporaire);
```

```
ingredient[i] = variable_temporaire;
```